

CAPÍTULO 1

FUNDAMENTOS DEL LENGUAJE C/C++



Lenguaje C/C++. Problemas, Casos y Proyectos

Índice del Capítulo 1

Introducción	4
Objetivos	4
1.1. Creación de un Programa en Lenguaje C/C++	5
1.1.1. Utilización de una Herramienta de Programación para el Lenguaje C/C++	5
1.1.2. Creación de un Proyecto	6
1.1.3. Añadiendo un Archivo .CPP al Proyecto	9
1.1.4. Escribiendo el Código del Programa	10
Problema 1.1. (Imprimiendo mensajes con la función cout)	10
Problema 1.2. (Imprimiendo mensajes con la función printf)	10
1.1.5. Uso de Comentarios en un Programa	11
1.1.6. Uso de Espacios en Blanco	12
Problema 1.3. (Uso de espacios en blanco)	12
Problema 1.4. (Uso de espacios en blanco)	13
1.1.7. Compilación, Enlazamiento y Ejecución	14
1.2. Estructura Básica de un Programa en C/C++	16
1.2.1. Directivas include	16
1.2.2. Espacios de nombre (Namespaces)	17
1.2.3. Directivas define o macros	17
Problema 1.5. (Aplicación de las directivas define o macros)	17
Problema 1.6. (Aplicación de las directivas define o macros)	18
1.2.4. La Función Principal	19
1.2.5. Estructura Básica de un Programa de Programación en C/C++	19
1.3. Funciones de Entrada y Salida de Datos	20
1.3.1. Función de Salida cout <<	20
1.3.2. Función de Entrada cin >>	20
Problema 1.7. (Aplicación de las funciones cout y cin)	20
1.3.3. Función de Salida printf()	21
1.3.4. Especificadores de Formato	22
1.3.5. Secuencias de Escape	22
Problema 1.8. (Aplicación de las secuencias de escape)	22
1.3.6. Función de Entrada scanf()	23
Problema 1.9. (Aplicación de las funciones printf y scanf)	23
1.3.7. Funciones de Utilidades Generales Estándar	24
1.4. Variables	24
1.4.1. Definición de Variable	25
1.4.2. Tipos de Datos en C/C++	25
1.4.3. Declaración y Definición de Variables	26
Problema 1.10. (Aplicación del uso de variables con las funciones cout y cin)	26
Problema 1.11. (Aplicación del uso de variables con las funciones printf y scanf)	27
1.4.4. Operador de Asignación Igual (=)	29
Problema 1.12. (Aplicación del operador de asignación igual utilizando cout y cin)	29
Problema 1.13. (Aplicación del operador de asignación igual utilizando printf y scanf)	31
1.4.5. Nombres de Variables	32
1.5. Operadores Aritméticos	33
1.5.1. Operadores Aritméticos Binarios	33
Problema 1.14. (Aplicación de los operadores aritméticos binarios utilizando cout y cin)	33
Problema 1.15. (Aplicación de los operadores aritméticos binarios utilizando printf y scanf)	34
1.5.2. El Operador de Módulo	35
1.5.3. Expresiones Aritméticas Compuestas	35
Problema 1.16. (Aplicación de las expresiones aritméticas compuestas utilizando cout y cin)	35
Problema 1.17. (Aplicación de las expresiones aritméticas compuestas utilizando printf y scanf)	37
1.5.4. Operadores Aritméticos Unarios	38
Problema 1.18. (Aplicación de los operadores aritméticos unarios utilizando cout y cin)	39
1.5.5. Prueba de Escritorio	40
Problema 1.19. (Aplicación de los operadores aritméticos unarios utilizando printf y scanf)	42
Problema 1.20. (Aplicación de los operadores aritméticos unarios con expresiones matemáticas complejas utilizando cout y cin)	44

Lenguaje C/C++. Problemas, Casos y Proyectos

Problema 1.21. (Aplicación de los operadores aritméticos unarios con expresiones matemáticas complejas utilizando printf y scanf)	45
1.5.6. Precedencia de Operadores Aritméticos	47
Problema 1.22. (Aplicación del uso de precedencia de operadores aritméticos con las funciones cout y cin)	48
Problema 1.23. (Aplicación del uso de precedencia de operadores aritméticos con las funciones printf y scanf)	49
1.6. Operadores Relacionales	50
Problema 1.24. (Aplicación de los operadores relacionales utilizando cout y cin)	50
Problema 1.25. (Aplicación de los operadores relacionales utilizando printf y scanf)	52
1.7. Operadores Lógicos	53
Problema 1.26. (Aplicación de los operadores lógicos utilizando cout y cin)	54
Problema 1.27. (Aplicación de los operadores lógicos utilizando printf y scanf)	55
1.8. Fórmulas Matemáticas	56
1.8.1. Funciones de la Librería cmath	56
Problema 1.28. (Aplicación del uso de funciones matemáticas con las funciones cout y cin)	57
Problema 1.29. (Aplicación del uso de funciones matemáticas con las funciones printf y scanf)	58
Problema 1.30. (Cálculo del área y perímetro de un cuadrado utilizando cout y cin)	59
Problema 1.31. (Cálculo del área y perímetro de un cuadrado utilizando printf y scanf)	60
Problema 1.32. (Cálculo del área y perímetro de un rectángulo utilizando cout y cin)	61
Problema 1.33. (Cálculo del área y perímetro de un rectángulo utilizando printf y scanf)	62
Problema 1.34. (Cálculo del área y perímetro de un círculo utilizando cout y cin)	63
Problema 1.35. (Cálculo del área y perímetro de un círculo utilizando printf y scanf)	64
Problema 1.36. (Cálculo de la hipotenusa y de los ángulos de un triángulo rectángulo utilizando cout y cin)	65
Problema 1.37. (Cálculo de la hipotenusa y de los ángulos de un triángulo rectángulo utilizando printf y scanf)	66
Problema 1.38. (Cálculo de la altura de un proyectil con una trayectoria parabólica)	67
1.9. Resumen	69
1.10. Test de Conocimientos	70
1.11. Problemas Propuestos	72
Referencias Bibliográficas	78



Introducción

El *Lenguaje C/C++* es un lenguaje poderoso de alto nivel que combina el paradigma de programación estructurado, con las eficiencias del bajo nivel, tales como la habilidad de manipular directamente la memoria. Por estas razones, el *lenguaje C/C++* ha sido adoptado como el lenguaje para desarrollar sistemas operativos tales como: Windows, Mac, LINUX, entre otros; bases de datos como Oracle, SQL Server, etc.; software para control industrial como PIC C, Ladder, LabVIEW; software para aplicaciones matemáticas como Matlab, Scilab; software para aplicaciones de escritorio como Microsoft Office, Open Office y un sinnúmero de aplicaciones de todo tipo.



Dennis Ritchie¹



Bjarne Stroustrup²

El creador del lenguaje C fue el científico de la computación Dennis Ritchie (1941-2011). A su creador le tomo tres años en desarrollarlo entre los años 1969 y 1972 en los Laboratorios Bell, como evolución del lenguaje B, a su vez basado en BCPL (Ritchie, D., Kernighan, B., 1988). En el año de 1979 el científico de la computación Bjarne Stroustrup lanza el lenguaje C++ con mecanismos de abstracción que permiten la manipulación de objetos y la combinación de los paradigmas de programación estructurada y orientada a objetos, lo que le constituye en un lenguaje híbrido (Stroustrup, B., 1997).

Objetivos

- Crear, compilar, enlazar y ejecutar programas básicos en C/C++.
- Entender la estructura básica de un programa en C/C++.
- Aprender el funcionamiento de las funciones básicas de entrada y salida de datos.
- Entender el concepto de variable y tipos de datos.
- Revisar los operadores aritméticos, relacionales y lógicos que maneja el lenguaje C/C++.
- Revisar las principales fórmulas matemáticas que maneja el lenguaje C/C++.
- Diseñar y crear programas en lenguaje C/C++ utilizando operadores aritméticos, relacionales y lógicos.

¹ Imagen tomada de: MUYLINUX. Dennis Ritchie y Ken Thompson ganan el Japan Prize 2011. Disponible en: <https://www.muylinux.com/2011/01/27/dennis-ritchie-y-ken-thompson-ganan-el-japan-prize-2011/>.

² Imagen tomada de: AZ QUOTES. Bjarne Stroustrup Quotes. Disponible en: https://www.azquotes.com/author/14260-Bjarne_Stroustrup.

1.1. Creación de un Programa en Lenguaje C/C++ utilizando Visual C++ .NET de Microsoft

Un programa es una lista de instrucciones debidamente estructuradas para que la computadora ejecute una serie de operaciones o cumpla un propósito. Una operación podría ser la suma de dos números, el perímetro y el área de un círculo, el volumen de un cilindro o la impresión de algunos datos en la pantalla de la computadora. (Ver Figura 1.1).

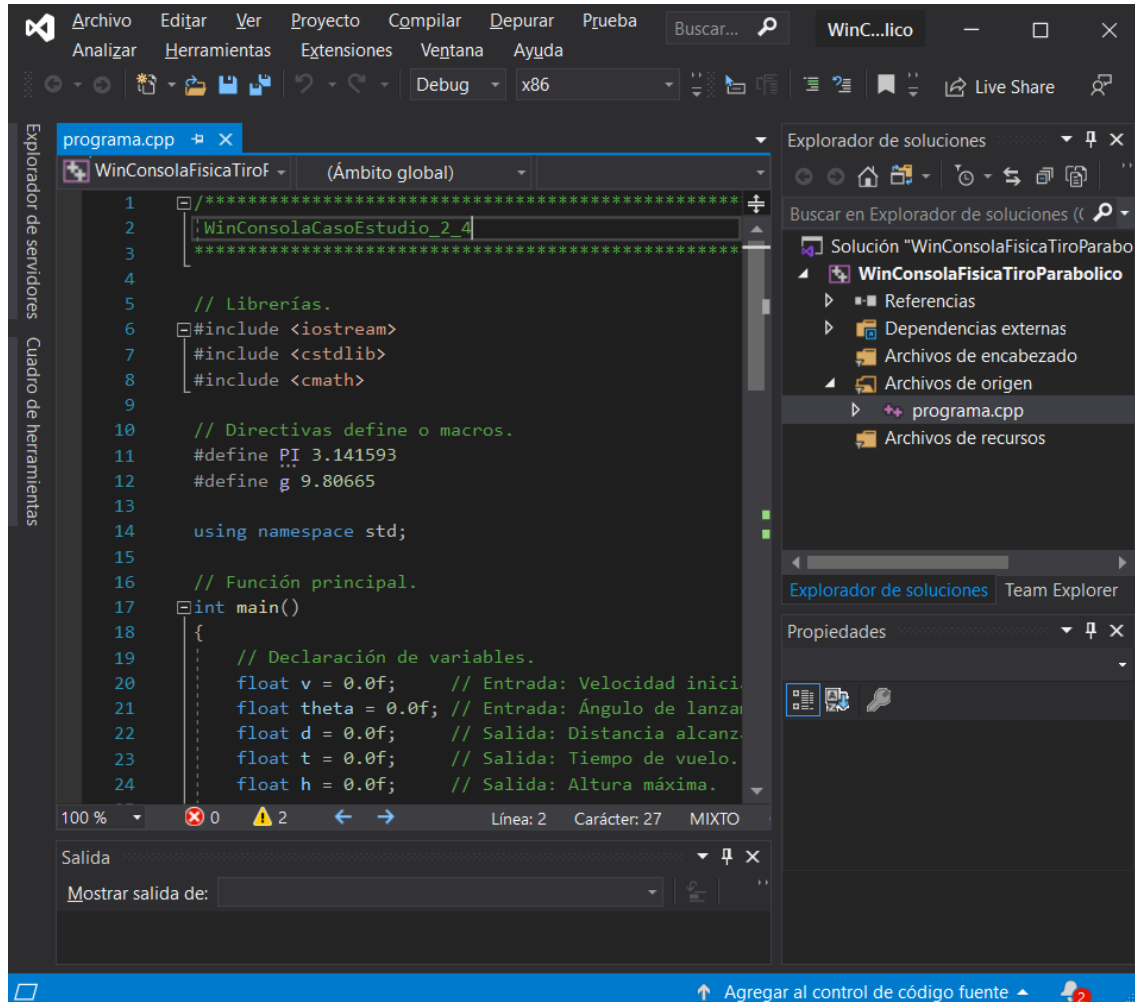


Figura 1.1. Ejemplo de un Programa en C/C++.

1.1.1. Utilización de una Herramienta de Programación para el Lenguaje C/C++

Existe una gran variedad de herramientas de software para programar en *lenguaje C*, tales como: *Visual C++ .NET*, *OnlineGDB*, *Borland C/C++*, *DEV C++*, *Code::Blocks*, entre otras. En la presente sección vamos a aprender paso a paso, como crear, compilar, enlazar y ejecutar un programa en *lenguaje C/C++* utilizando *Visual C++ .NET* que viene en el paquete de desarrollo de *Visual Studio Community* (edición 2019 o 2022), que es de libre distribución para plataformas Microsoft. (Ver Figura 1.2).



Figura 1.2. Pantalla de Presentación de la Herramienta de Desarrollo Visual Studio.

1.1.2. Creación de un Proyecto

Después de cargarse la pantalla de presentación de Visual Studio, aparece el cuadro de diálogo de *Tareas iniciales*, donde se debe seleccionar la tarea: *Crear un proyecto*, como se muestra en la Figura 1.3.

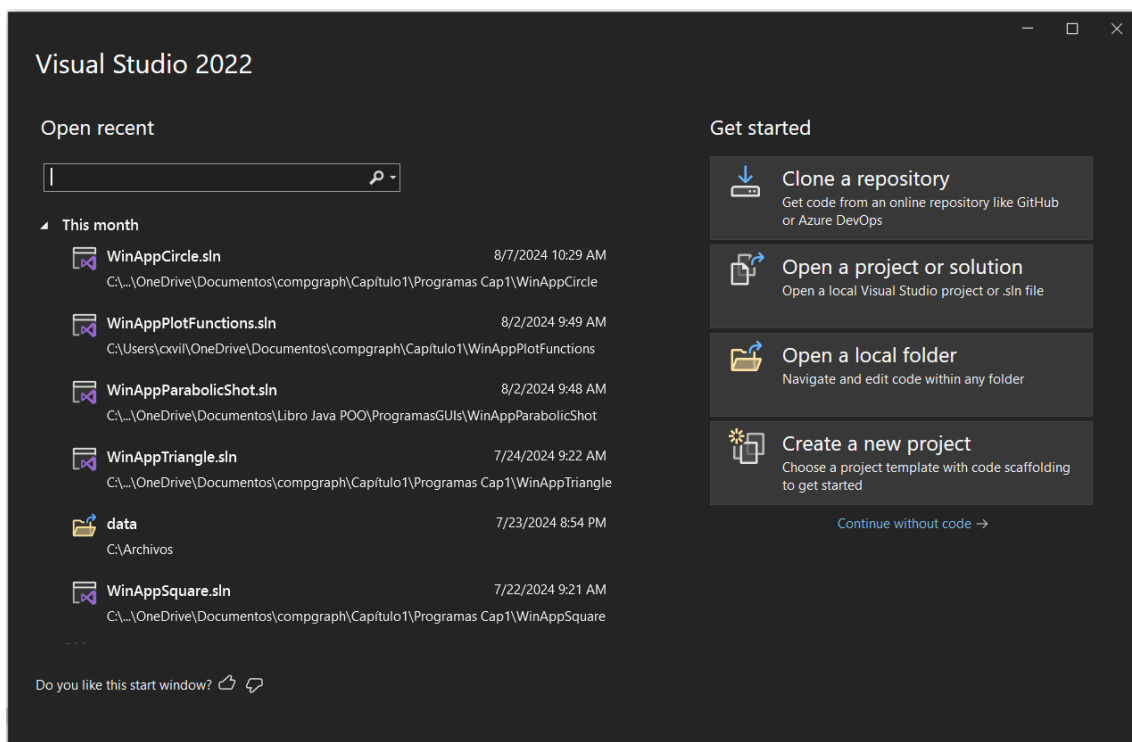


Figura 1.3. Tareas iniciales de Visual Studio.

Lenguaje C/C++. Problemas, Casos y Proyectos

En el cuadro de diálogo de *Crear un proyecto* se debe seleccionar el **IDE Visual C++** de la lista de selección del lado izquierdo, ubicado debajo de la opción de *Buscar plantillas*, como se muestra en la Figura 1.4.

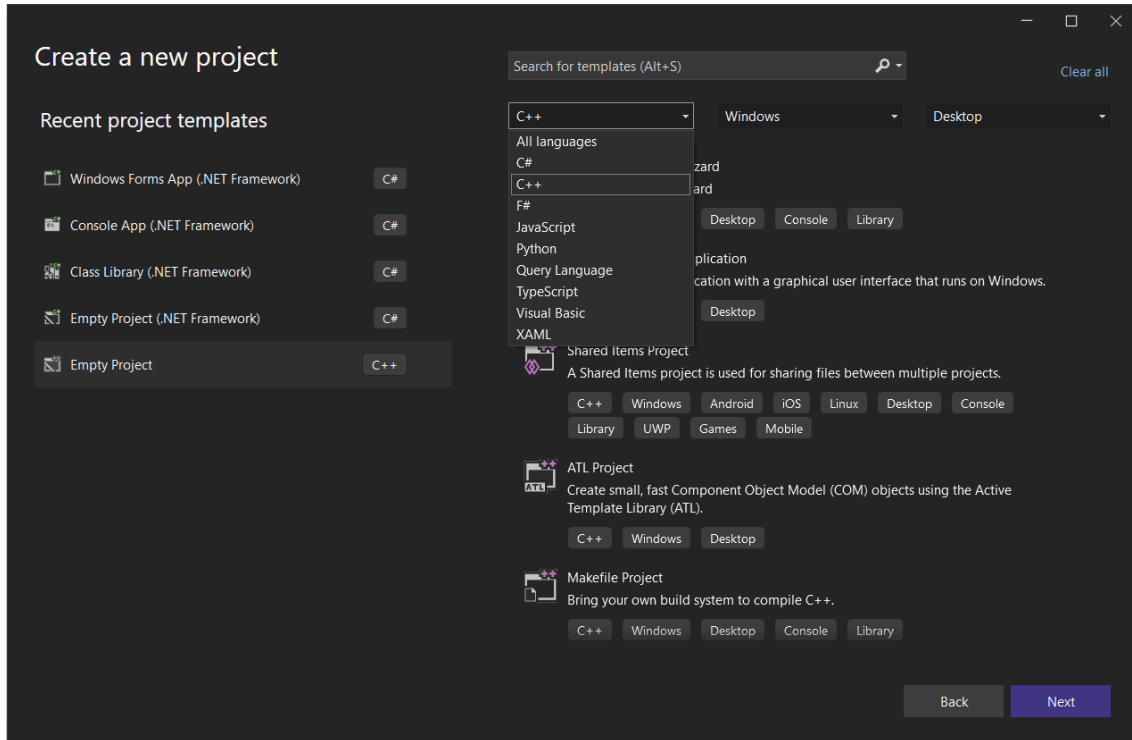


Figura 1.4. Selección del IDE Visual C++ .NET para el Lenguaje C/C++.

A continuación, se debe seleccionar la opción de **Proyecto vacío** (C++, Windows, Consola), para empezar de cero con C++ para Windows. (Ver Figura 1.5).

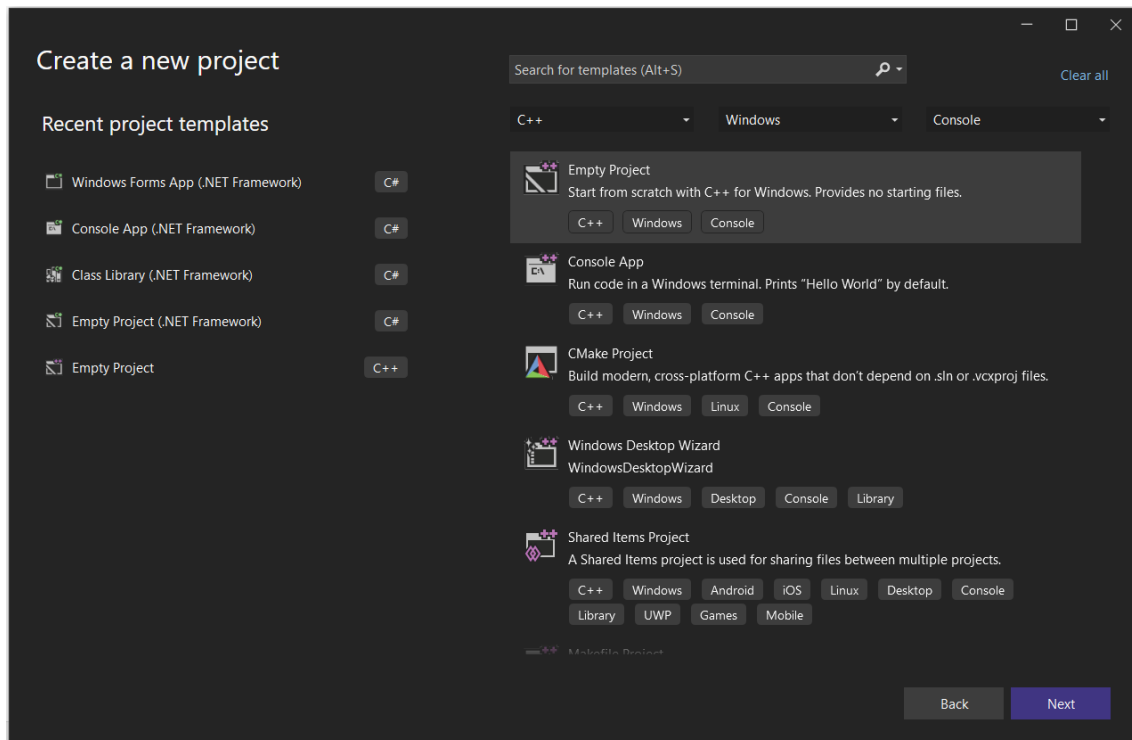


Figura 1.5. Seleccionar Proyecto vacío (C++, Windows, Consola).

Lenguaje C/C++. Problemas, Casos y Proyectos

En el cuadro de diálogo de **Configure su nuevo proyecto**, ingrese un nombre que usted escoja para el proyecto como en este caso **WinConsolaPrograma_1_1** y seleccione la ubicación del proyecto donde se va a almacenar como el siguiente **C:\Users\cxvil\OneDrive\Documentos\Visual Studio 2022\Code Snippets\Visual C++**. Luego presione el botón de **Crear**, como se puede ver en la Figura 1.6.

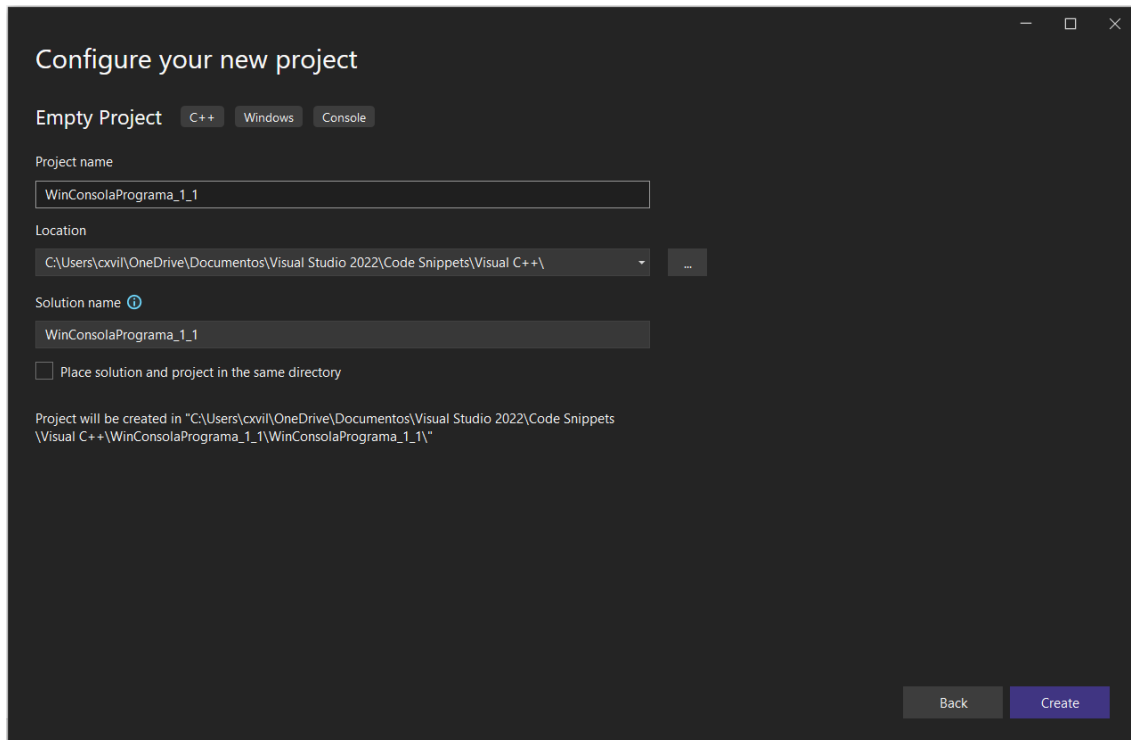


Figura 1.6. Cuadro de diálogo de Configure su nuevo proyecto.

En este punto nosotros hemos creado exitosamente un proyecto en Lenguaje C/C++ correspondiente a un *Proyecto de aplicación de consola*, como se puede ver en la Figura 1.7. El siguiente paso es añadir un archivo de código fuente (.CPP) al proyecto, es decir, un archivo con el cual nosotros a continuación escribiremos nuestro código en lenguaje C/C++.

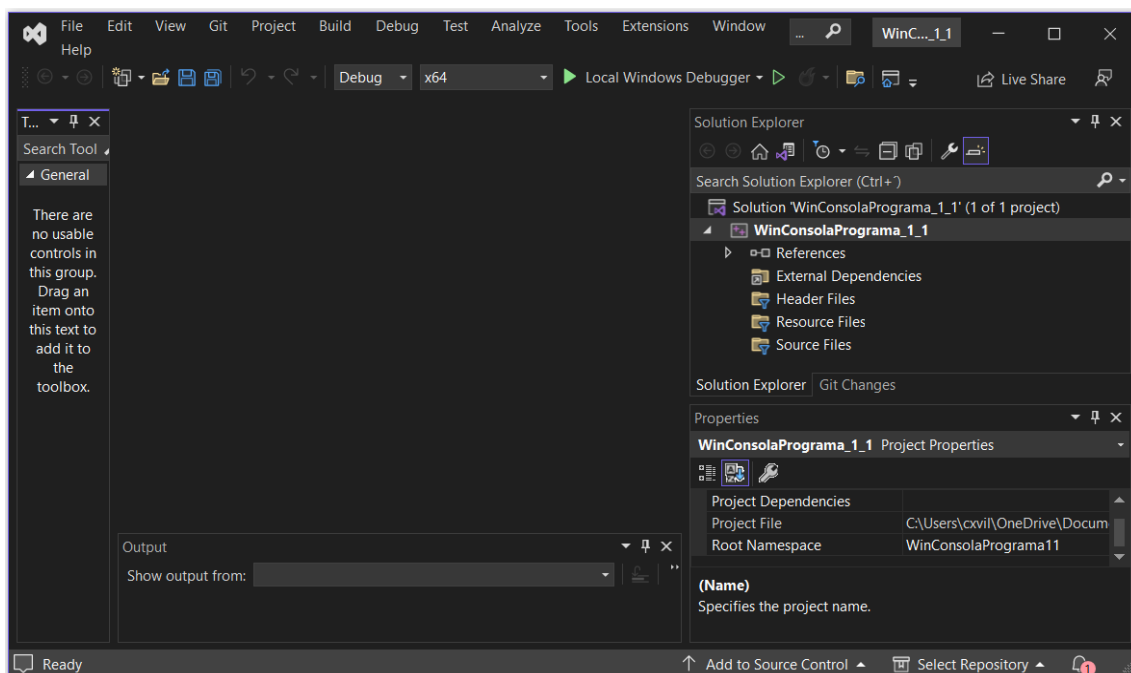


Figura 1.7. Proyecto de aplicación de consola.

1.1.3. Añadiendo un Archivo .CPP al Proyecto

Para añadir un archivo .CPP a su proyecto, seleccione la carpeta *Archivos de origen* y luego haga un clic derecho sobre esta carpeta y seleccione *Agregar -> Nuevo elemento...* como se muestra en la Figura 1.8.

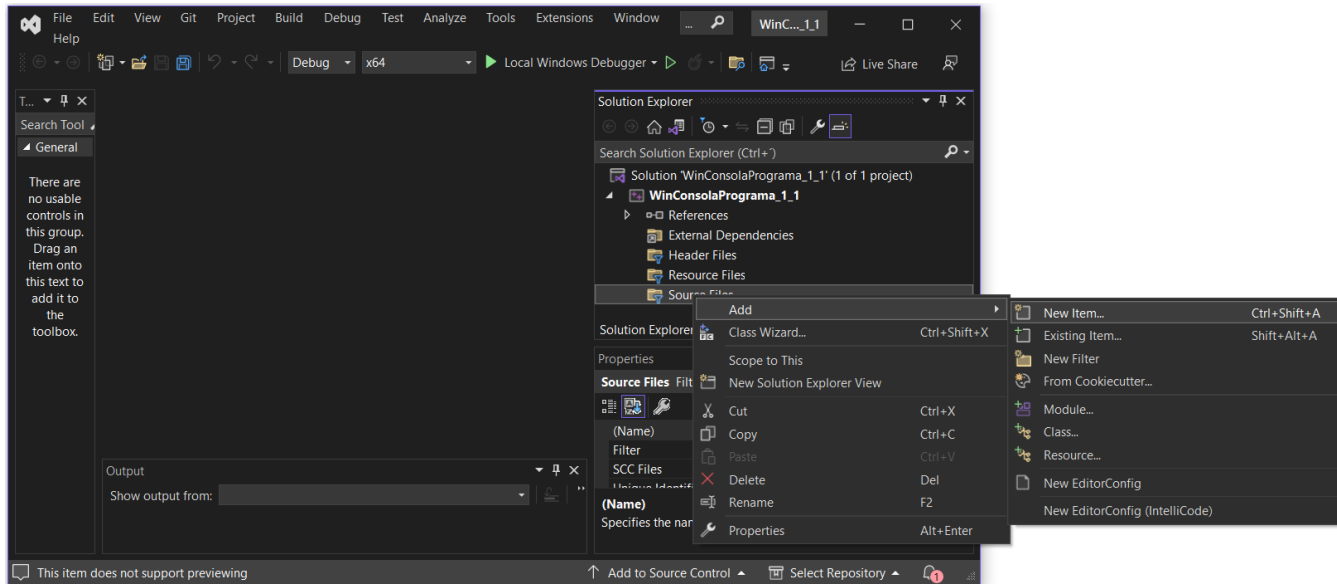


Figura 1.8. Agregando un nuevo elemento al proyecto.

A continuación, aparece la caja de diálogo de *Agregar nuevo elemento*, como se puede ver en la Figura 1.9. Aquí se debe seleccionar un *Archivo C++ (.cpp)* y se debe colocar un nombre al archivo .CPP, como en este caso **programa.cpp** y presione el botón de *Agregar*. Un archivo .CPP vacío aparecerá automáticamente y será abierto en *Visual C++ .NET*.

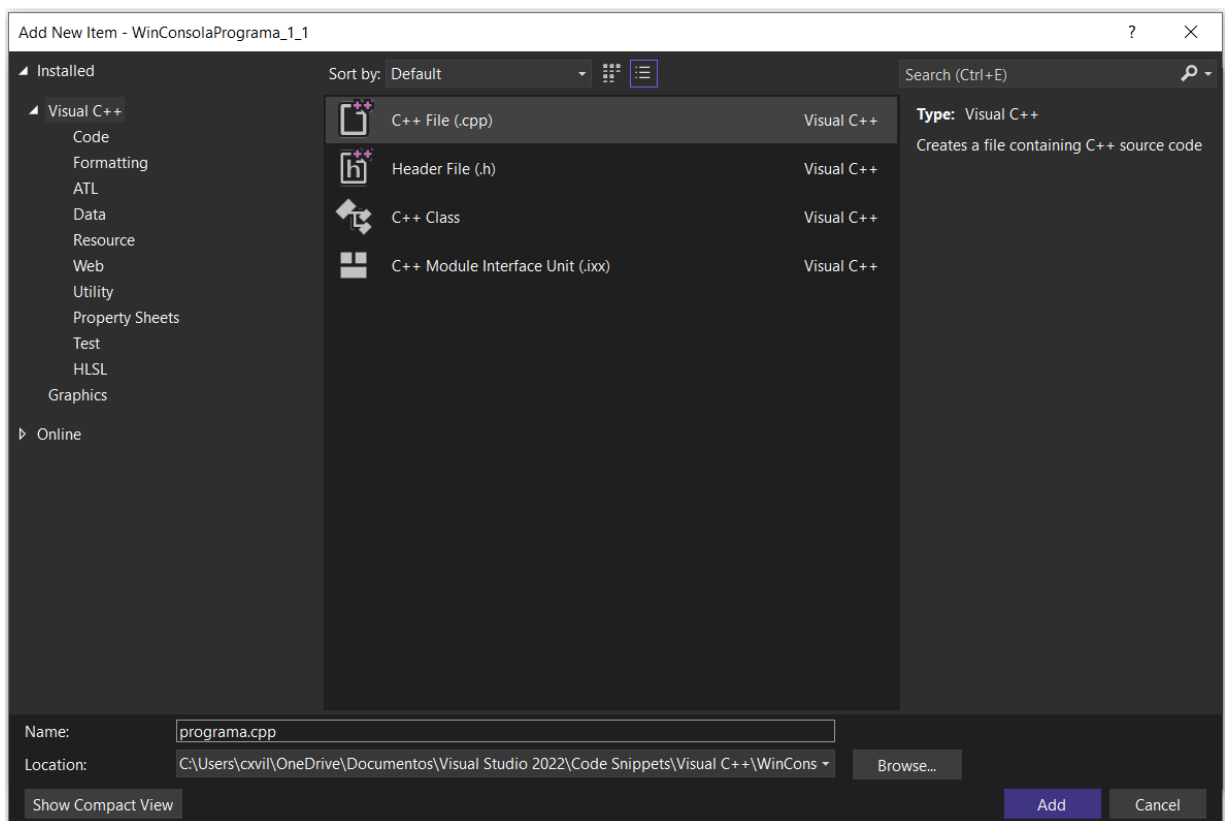


Figura 1.9. Cuadro de Diálogo de Agregar nuevo elemento.

1.1.4. Escribiendo el Código del Programa

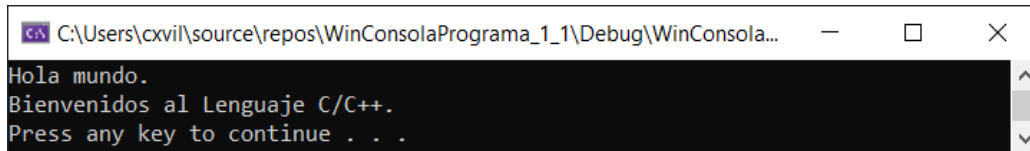
Usted a continuación verá un archivo .CPP en blanco, donde nosotros podemos comenzar a escribir nuestro código del programa. Escriba o copie el código siguiente en lenguaje C/C++, exactamente tal y como está, dentro de su archivo .CPP.

Problema 1.1: Escribir un programa que imprima un mensaje de "Hola mundo." y otro que diga "Bienvenidos al Lenguaje C/C++.", utilizando la función de salida de datos "cout".

Programa 1.1: Código del programa.

```
/******  
WinConsolaPrograma_1_1  
******/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // Función cout, que imprime mensajes de texto.  
    cout << "Hola mundo." << endl;  
    cout << "Bienvenidos al Lenguaje C/C++." << endl;  
  
    // Incorporar una pausa en el programa.  
    system("pause");  
    return 0;  
}
```

Salida 1.1: Salida del programa.



Como se puede ver en este programa, nosotros desplegamos el texto: "Hola mundo." en la consola mediante la función cout y el operador <<, seguido de un salto de línea utilizando el comando endl, como se indica en la siguiente instrucción:

```
cout << "Hola mundo." << endl;
```

Luego nosotros desplegamos el texto: "Bienvenidos al Lenguaje C/C++." en la consola mediante la función cout y el operador <<, seguido de un salto de línea utilizando el comando endl, como se indica en la siguiente instrucción:

```
cout << "Bienvenidos al Lenguaje C/C++." << endl;
```

Finalmente incorporamos una pausa en el programa utilizando la función:

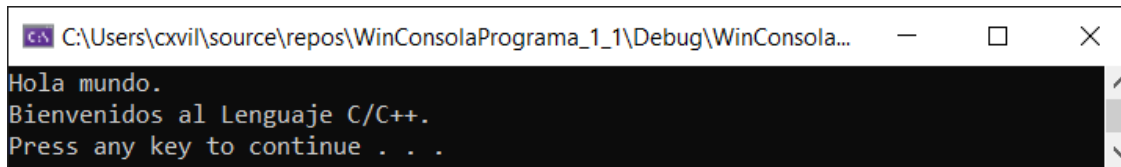
```
system("pause");
```

Problema 1.2: Escribir un programa que imprima un mensaje de "Hola mundo" y otro que diga "Bienvenidos al Lenguaje C/C++.", utilizando la función de salida de datos "printf".

Programa 1.2: Código del programa.

```
/* ****  
WinConsolaPrograma_1_2  
**** */  
  
// Librerías.  
#include <stdio.h>  
#include <stdlib.h>  
  
// Función principal.  
int main()  
{  
    // Función printf(), que imprime mensajes de texto.  
    printf("Hola mundo.\n");  
    printf("Bienvenidos al Lenguaje C/C++.\n");  
  
    // Incorporar una pausa en el programa.  
    system("pause");  
    return 0;  
}
```

Salida 1.2: Salida del programa.



Como se puede ver en este programa, nosotros desplegamos el texto: "Hola mundo." en la consola mediante la función `printf`, seguido de un salto de línea utilizando la secuencia de escape `"\n"`, como se indica en la siguiente instrucción:

```
printf("Hola mundo.\n");
```

Luego nosotros desplegamos el texto: "Bienvenidos al Lenguaje C/C++." en la consola mediante la función `printf`, seguido de un salto de línea utilizando la secuencia de escape `"\n"`, como se indica en la siguiente instrucción:

```
printf("Bienvenidos al Lenguaje C/C++.\n");
```

Finalmente incorporamos una pausa en el programa utilizando la función:

```
system("pause");
```

1.1.5. Uso de Comentarios en un Programa

El lenguaje C/C++, maneja dos tipos de comentarios:

- Quando se quiere comentar más de una línea de código, se utiliza la instrucción `/* ... */`, como consta en el Programa 1.1, donde se tiene el siguiente ejemplo:

```
/* ****  
WinConsolaPrograma_1_1  
**** */
```

- b) Cuando se quiere comentar una sola línea de código, se utiliza la instrucción '//', como consta en el Programa 1.1, donde se tienen los siguientes ejemplos:

```
// Librerías.  
// Función principal.  
// Función cout, que imprime mensajes de texto.  
// Incorporar una pausa en el programa.
```

Los comentarios son textos o cadenas de caracteres que son ignoradas por el compilador. Cuando el compilador compila el programa, se salta cualquier comentario y no lo toma en cuenta.

Escribir comentarios claros resulta especialmente importante cuando se trabaja en equipos de trabajo, donde otros programadores necesitarán leer y modificar su código. Los “buenos comentarios” son claros, concisos, y fáciles de entender, contrario a los “malos comentarios” que son inconsistentes, fuera de fecha, ambiguos, vagos, superfluos y no los puede entender cualquiera. En el libro de “The C++ Programming Language. Special Edition” de Bjarne Stroustrup, el autor menciona que: “Escribir buenos comentarios puede ser tan difícil como escribir el programa. Esto es un arte que vale la pena cultivarlo” (Stroustrup, B., 1997).

1.1.6. Uso de Espacios en Blanco

El programa 1.1 tiene en varias partes del programa espacios en blanco. Por ejemplo, entre los ‘comentarios iniciales’ y las ‘directivas include’, se tiene una línea en blanco. Las líneas en blanco y los espacios se conocen como ‘espacios en blanco’ (white space). El compilador ignora el momento de la compilación, todos los espacios en blanco. Se puede tener múltiples espacios en blanco, entre todas las instrucciones de C/C++ (Granizo, E., 2016). A continuación, se redefine el programa 1.1 con varios espacios en blanco, en el programa 1.3. También, se redefine el programa 1.2 con varios espacios en blanco, en el programa 1.4.

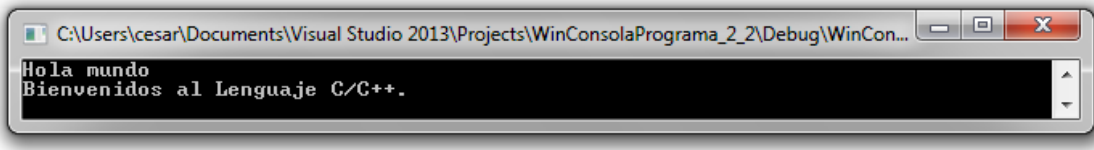
Problema 1.3: Reescribir el programa 1.1 de tal manera que utilice varios espacios en blanco.

Programa 1.3. Código del programa.

```
/*  
*****  
WinConsolaPrograma_1_3  
*****  
*/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
  
using namespace std ;  
  
// Función principal.  
int  
main()  
{  
    // Función cout, que imprime mensajes de texto.  
    cout << "Hola mundo" << endl ;  
    cout << "Bienvenidos al Lenguaje C/C++." << endl ;  
    // Incorporar una pausa en el programa.  
    system("pause") ;  
    return 0 ;  
}
```

Lenguaje C/C++. Problemas, Casos y Proyectos

Salida 1.3: Salida del programa.



Luego se puede compilar este programa y se va a poder observar que éste trabaja correctamente.

Nótese que no hay espacios dentro de una cadena de caracteres o texto, ya que los espacios entre las cadenas de caracteres no son ignorados por el compilador. El lenguaje C/C++ no permite que las palabras clave (keywords) sean rotas o separadas en sub-palabras.

Por ejemplo, no se puede escribir la instrucción:

```
cout << "Hola mundo" << endl;
```

de la siguiente manera:

```
co  ut <      < "Hola mundo" <      < en  dl;
```

Los símbolos y palabras reservadas que tienen un significado actual deben ser mantenidos intactos, así como originalmente fueron definidos.

Finalmente se recomienda utilizar tabulación entre instrucción e instrucción para organizar de mejor manera el código del programa. A este proceso se conoce como indentación (indentation) y ayuda a corregir posibles errores dentro de un programa y a realizar mantenimiento al mismo.

Problema 1.4: Reescribir el programa 1.2 de tal manera que utilice varios espacios en blanco.

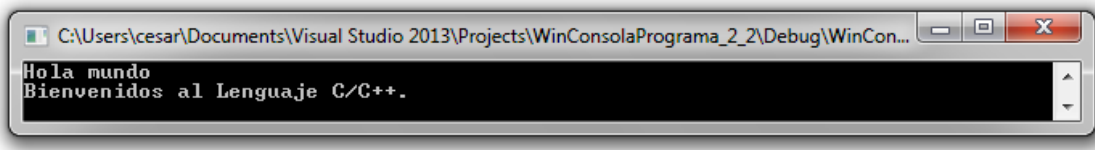
Programa 1.4. Código del programa.

```
/* *****  
WinConsolaPrograma_1_4  
***** */  
  
// Librerías.  
#include <stdio.h>  
  
#include <stdlib>  
  
// Función principal.  
int  
main()  
{  
    // Función printf(), que imprime mensajes de texto.  
  
    printf          ("Hola mundo.\n")          ;  
    printf          ("Bienvenidos al Lenguaje C.\n")      ;  
  
    // Incorporar una pausa en el programa.  
  
    system("pause")          ;  
    return          0          ;  
}
```



Lenguaje C/C++. Problemas, Casos y Proyectos

Salida 1.4: Salida del programa.



Luego se puede compilar este programa y se va a poder observar que éste trabaja correctamente.

Nótese que no hay espacios dentro de una cadena de caracteres o texto, ya que los espacios entre las cadenas de caracteres no son ignorados por el compilador. El lenguaje C/C++ no permite que las palabras clave (keywords) sean rotas o separadas en sub-palabras.

Por ejemplo, no se puede escribir la instrucción:

```
printf("Hola mundo.\n");
```

de la siguiente manera:

```
pr in tf("Hola mundo.\n");
```

Los símbolos y palabras reservadas que tienen un significado actual deben ser mantenidos intactos así como originalmente fueron definidos.

1.1.7. Compilación, Enlazamiento y Ejecución

Después de que el código de un programa en lenguaje C/C++ está completado, éste debe ser traducido a lenguaje máquina que es el lenguaje que entiende la computadora. Para ser ejecutado un programa se cumplen con dos pasos en este proceso de traducción:

- 1) Compilación
- 2) Enlazamiento

En el primer paso que es la *compilación*, el *compilador* compila cada *archivo de código fuente* (.CPP) de un proyecto. Hay casos de proyectos complejos que contienen más de un archivo de código fuente y se generan los *archivos objeto* (.OBJ) por cada archivo de código fuente. Se dice que un archivo objeto contiene el código objeto. (Ver Figura 1.10).

En el segundo paso que es el *enlazamiento*, el *enlazador* combina todos los archivos objeto y cualquier otro *archivo de librería* (.LIB), para producir un *archivo ejecutable* (.EXE). Este archivo ejecutable es el que correrá en una plataforma (Luna, F., 2004).

Para compilar un programa en *Visual C++ .NET*, como, por ejemplo, el *Programa 1.1*, se debe seleccionar de la barra de menús la opción de *Depurar -> Iniciar depuración*. Una ventana de resultados de la compilación se desplegará como la que se muestra en la Figura 1.11.

Observe que cuando no se tiene ningún error, ni ninguna advertencia (warning), esto significa que hemos escrito código legal en C/C++ y, por lo tanto, nuestro programa ha sido compilado exitosamente.

En cambio, cuando se escribe código ilegal en C/C++ que significa que el programa tiene errores como, por ejemplo, puntuación incorrecta; así es el caso de que en el programa falte un punto y coma ';' se produce un error y a continuación el siguiente mensaje se presenta: error C2146: error de sintaxis: falta ';' delante del identificador 'cout'. Usted puede utilizar este mensaje de error para localizar con precisión el problema dentro de su código del programa y corregirlo. (Ver Figura 1.12).

1.2. Estructura Básica de un Programa en C/C++

En general la estructura básica de un programa en C/C++ (Granizo, E., 2016), consta de las siguientes partes:

- Directivas include.
- Directivas define o macros.
- La función principal.

La Figura 1.13 muestra la estructura básica de un programa en lenguaje C/C++ donde constan las librerías de encabezado (directivas include, directivas define) y la función principal con la lógica del programa.

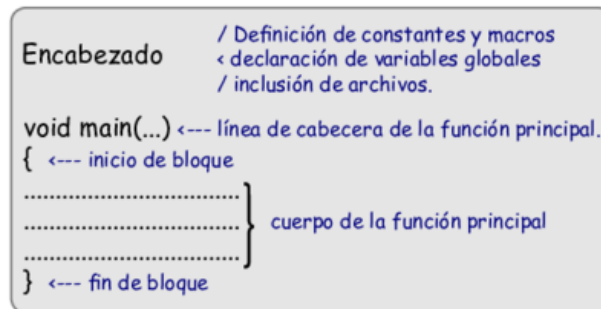


Figura 1.13. Estructura básica de un programa en Lenguaje C/C++.

1.2.1. Directivas include

En el programa 1.1 se necesitó utilizar algo de código que no fue escrito por nosotros mismos. En este curso, la mayor parte de este “código útil”, representado por funciones, lo encontraremos en las librerías de encabezado de C/C++ (Luna, F., 2004), que es un conjunto de código que se envía al compilador enlazado con el programa. Algunos ejemplos de este “código útil” se mencionan a continuación en la Tabla 1.1:

Tabla 1.1: Librerías básicas de C/C++.

Librería	Descripción
<code>#include <iostream></code>	Librería de Entrada y Salida de Flujo de Datos (Input/Output Stream). Esta librería es un componente de la biblioteca estándar (STL) del lenguaje de programación C++ que es utilizado para operaciones de entrada y salida.
<code>#include <stdio.h></code>	Librería de Encabezado Estándar de Entrada y Salida (Standard Input & Output Library Header). Esta librería tiene funciones para imprimir (output) y leer datos (input) desde la ventana de la consola.
<code>#include <cstdlib></code>	Librería de Utilidades Generales Estándar (C Standard General Utilities Library). Esta librería tiene funciones para gestión de memoria dinámica, generación de números aleatorios, comunicación con el ambiente, búsqueda de datos, ordenamiento de datos y control de pausas del sistema, entre otras.
<code>#include <cmath></code>	Librería de Funciones Matemáticas (C Math Library). Esta librería tiene funciones para realizar operaciones matemáticas básicas, tales como: seno(), coseno(), tangente(), etc.
<code>#include <ctime></code>	Librería de Tiempo (C Time Library). Esta librería contiene funciones para manipular y formatear la fecha y hora del sistema.
<code>#include <string></code>	Librería de Cadenas de Caracteres (String Library). Esta librería contiene la definición de macros, constantes, funciones y tipos de utilidad para trabajar con cadenas de caracteres y algunas operaciones de manipulación de memoria.

Lenguaje C/C++. Problemas, Casos y Proyectos

En el Programa 1.1, se invoca a dos directivas include:

```
#include <iostream>
#include <cstdlib>
```

En el Programa 1.2, se invoca a dos directivas include:

```
#include <stdio.h>
#include <cstdlib>
```

Cada una de estas directivas van encerradas entre los símbolos de “menor que” y “mayor que” y le informa al compilador que debe tomar estos códigos e incluirlos en nuestro archivo .CPP.

Nótese que al incluir los archivos de las librerías de encabezado de C/C++, su programa se enlaza a estos archivos de librerías estándar; sin embargo, esto se lo hace automáticamente cuando el proyecto es creado en C/C++.

1.2.2. Espacios de nombre (Namespaces)

Los espacios de nombre son al código como las carpetas son a los archivos (Luna, F., 2004). Así como las carpetas son utilizadas para organizar grupos de archivos relacionados y prevenir que el nombre de un archivo entre en conflicto, los espacios de nombre (namespaces) son utilizados para organizar grupos de código relacionado y prevenir que el nombre del código entre en conflicto. Por lo tanto, la librería completa estándar de C/C++ es organizada en el espacio de nombre ‘std’ y debido a esto se debe incluir la cláusula:

```
using namespace std;
```

1.2.3. Directivas define o macros

Algunas veces se puede necesitar utilizar sentencias de preprocesador o macros que se crean con un nombre simbólico (identificador) y esta macro levanta algún conjunto de código (Luna, F., 2004). Por ejemplo, en el programa 1.1 se pudo haber definido una macro llamada IMPRIMIRMENSAJEHOLA, la cual ejecuta la sentencia: `cout << "Hola mundo" << endl;` como se muestra a continuación:

```
// Definir una macro para imprimir un mensaje de Hola mundo.
#define IMPRIMIRMENSAJEHOLA cout << "Hola mundo" << endl;
```

A continuación, se redefine nuevamente el programa 1.1 con directivas define o macros en el programa 1.5.

Problema 1.5: Reescribir el programa 1.1 de tal manera que utilice directivas define o macros.

Programa 1.5. Código del programa.

```
/* *****
WinConsolaPrograma_1_5
***** */

// Librerías.
#include <iostream>
#include <cstdlib>

using namespace std;

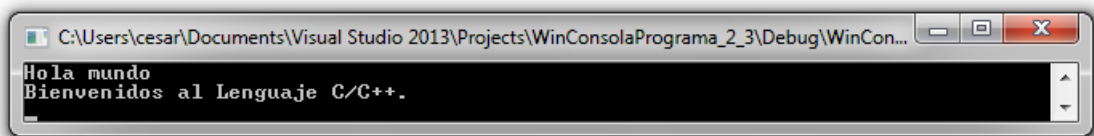
// Directivas define o macros.
```

Lenguaje C/C++. Problemas, Casos y Proyectos

```
// Definir una macro para imprimir un mensaje de Hola mundo.
#define IMPRIMIRMENSAJEHOLA cout << "Hola mundo" << endl;
// Definir una macro para imprimir un mensaje de Bienvenidos al Lenguaje C.
#define IMPRIMIRMENSAJEBIENVENIDOS cout << "Bienvenidos al Lenguaje C/C++." << endl;
// Definir una macro para incorporar una pausa en el programa.
#define INCORPORARPAUSA system("pause"); return 0;

// Función principal.
int main()
{
    IMPRIMIRMENSAJEHOLA
    IMPRIMIRMENSAJEBIENVENIDOS
    INCORPORARPAUSA
}
```

Salida 1.5: Salida del programa.



Nótese que el carácter punto y coma (;) ya está incluido en la definición de la macro, por lo que no es necesario colocar otro punto y coma (;) en la llamada a la macro.

Cuando el compilador compila el código fuente y encuentra una macro, internamente se reemplaza el símbolo de la macro con el código que fue definida.

A continuación, se redefine nuevamente el programa 1.2 con directivas define o macros en el programa 1.6.

Problema 1.6: Reescribir el programa 1.2 de tal manera que utilice directivas define o macros.

Programa 1.6. Código del programa.

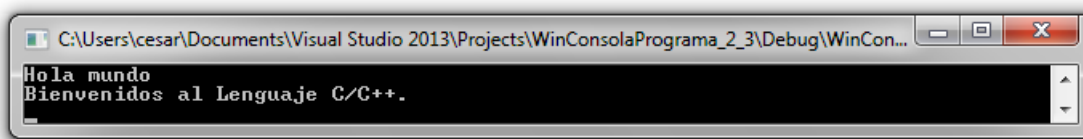
```
/*
*****
WinConsolaPrograma_1_6
*****
*/

// Librerías.
#include <stdio.h>
#include <stdlib.h>

// Directivas define o macros.
// Macro que imprime un mensaje de Hola mundo.
#define IMPRIMIRMENSAJEHOLA printf("Hola mundo.\n");
// Macro que imprime un mensaje de Bienvenidos al Lenguaje C.
#define IMPRIMIRMENSAJEBIENVENIDOS printf("Bienvenidos al Lenguaje C/C++.\n");
// Definir una macro para incorporar una pausa en el programa.
#define INCORPORARPAUSA system("pause"); return 0;

// Función principal.
int main()
{
    IMPRIMIRMENSAJEHOLA
    IMPRIMIRMENSAJEBIENVENIDOS
    INCORPORARPAUSA
}
```

Salida 1.6: Salida del programa.



Nótese que el carácter punto y coma (;) ya está incluido en la definición de la macro, por lo que no es necesario colocar otro punto y coma (;) en la llamada a la macro.

Cuando el compilador compila el código fuente y encuentra una macro, internamente se reemplaza el símbolo de la macro con el código que fue definida.

1.2.4. La Función Principal

La función `main(...)` es especial porque es el punto de entrada para cada programa en C/C++, es decir, cada programa en C/C++ comienza ejecutando su `main`. Consecuentemente, cada programa en C/C++ debe tener una función `main` (Joyanes, L., Zahonero, I., 2005). El código correspondiente a la función `main` va encerrado entre llaves y se lo conoce como el cuerpo de la función o la definición de la función. Estos conceptos y otros serán profundizados en el Capítulo 3, referente a Funciones.

En C/C++, las llaves `{}` son utilizadas para definir el inicio y el fin de una unidad de código lógico. Por ejemplo, en el programa 1.1, la llave abierta '{', denota el inicio de un código de bloque, y la llave cerrada '}', denota el fin del código de bloque. Un par de llaves deben siempre ir juntas en la función principal. En la Figura 1.14 se muestra un logo de la función principal y lo que ella significa.



Figura 1.14. Logo de la función principal en C/C++.

1.2.5. Estructura Básica de un Problema de Programación en C/C++

La Ingeniería de Software y la Metodología de Investigación Científica se puede aplicar de manera conjunta para la solución de problemas adaptados al desarrollo de software, donde desde el punto de vista pedagógico, la Metodología del Aprendizaje Basado en Problemas (Problem-Based Learning - PBL) es la más recomendable como parte del aprendizaje activo en el desarrollo de habilidades y la adquisición de conocimientos actualizados y relevantes por parte del estudiante, a través del planteamiento y la solución de problemas con diferentes niveles de dificultad (Zambrano, M., et al., 2021). En este capítulo se van a aplicar tres fases en la resolución de problemas de programación, considerando los aspectos de la ingeniería y el método científico dentro de cada fase.

1. **Planteamiento del Problema:** En esta fase, se plantea claramente el enunciado del problema que se desea resolver.

2. **Codificación del Programa:** En esta fase, se escribe el código del programa que permite automatizar un problema lógico-matemático o de cualquier área de las ciencias básicas como la Física, la Química, la Biología y las Matemáticas.
3. **Pruebas del Programa:** En esta fase, se verifica la validez de la solución del problema mediante la ejecución correcta del programa.

1.3. Funciones de Entrada y Salida de Datos

El archivo de la librería de encabezado 'iostream' contiene un sinnúmero de funciones para entrada y salida de datos. Entre las funciones más importantes para esta sección están la función 'cout <<' que es una función de salida de datos; y la función 'cin >>' que es una función para entrada de datos.

El archivo de la librería de encabezado "stdio.h" también contiene un sinnúmero de funciones para entrada y salida de datos. Entre las funciones más importantes para esta sección están la función printf() que es una función de salida de datos; y la función scanf() que es una función para entrada de datos. En la Figura 1.15 se muestra un ícono que representa a las funciones de entrada y salida de datos.

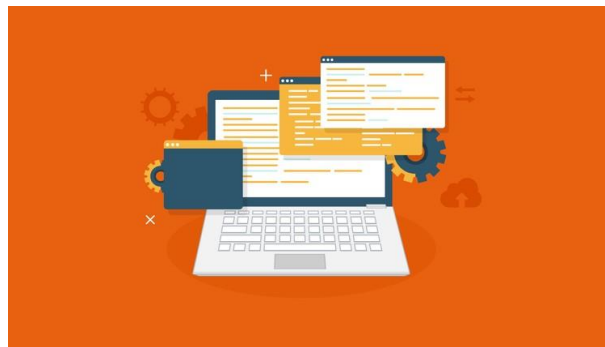


Figura 1.15. Ícono que representa las funciones de entrada y salida de datos.

1.3.1. Función de Salida cout <<

El programa 1.1 es capaz de desplegar datos desde la ventana de la consola. Al examinar el código del programa 1.1 se observa que se invoca dos veces a la función 'cout <<' para imprimir mensajes de texto. Entonces, la función 'cout <<' permite imprimir cadenas de caracteres o texto (Luna, F., 2004).

El símbolo '<<' se lo conoce como operador de inserción y toma sentido cuando se considera que este símbolo es utilizado para insertar una salida dentro de un flujo de salida de datos.

1.3.2. Función de Entrada cin >>

La función 'cin >>', es una función de entrada de datos por consola, que lee todos los tipos de datos definidos en C/C++, convirtiendo automáticamente al formato interno apropiado de acuerdo a los diferentes tipos de datos (Luna, F., 2004).

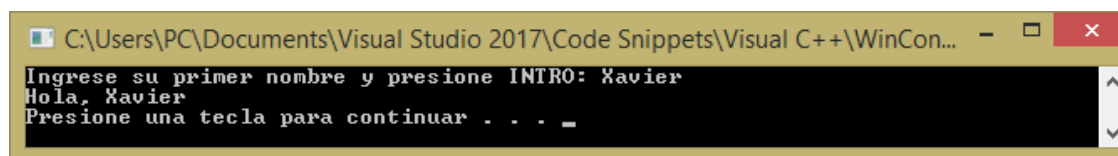
El símbolo '>>' se lo conoce como operador de extracción y toma sentido cuando se considera que este símbolo es utilizado para extraer una entrada dentro de un flujo de entrada de datos.

Problema 1.7: Escribir un programa que permita preguntarle al usuario su primer nombre y donde luego la computadora responda Hola usuario XYZ.

Programa 1.7: Código del programa.

```
/******  
WinConsolaPrograma_1_7  
******/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
#include <string>  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // Declaración de una variable.  
    string primerNombre = "";  
  
    // Leer el primer nombre de un usuario.  
    cout << "Ingrese su primer nombre y presione INTRO: "; // Salida.  
    cin >> primerNombre;                                     // Entrada.  
  
    // Imprimir un mensaje de Hola e imprimir el contenido de la  
    // variable primerNombre.  
    cout << "Hola, " << primerNombre << endl; // Salida.  
  
    // Incorporar una pausa en el programa.  
    system("pause");  
    return 0;  
}
```

Salida 1.7: Salida del programa.



Como se puede ver en este programa, nosotros desplegamos el texto: “Ingrese su primer nombre y presione INTRO: ” en la consola con la siguiente instrucción:

```
cout << "Ingrese su primer nombre y presione INTRO: ";
```

Luego nosotros le pedimos al usuario que ingrese su primer nombre con la siguiente instrucción:

```
cin >> primerNombre;
```

Finalmente nosotros desplegamos el mensaje “Hola, ” seguido de un valor string almacenado en la variable primerNombre y seguido de un salto de línea “endl”, con la siguiente instrucción:

```
cout << "Hola, " << primerNombre << endl;
```

1.3.3. Función de Salida printf()

El programa 1.2 es capaz de desplegar datos desde la ventana de la consola. Al examinar el código del programa 1.2 se observa que se invoca dos veces a la función ‘printf()’ para imprimir

Lenguaje C/C++. Problemas, Casos y Proyectos

mensajes de texto. Entonces, la función 'printf()' permite imprimir cadenas de caracteres o texto. El prototipo o la declaración de la función 'printf()' es el siguiente:

```
int printf(const char *formato [, argumentos, ...]);
```

La función 'printf()', como se puede ver en la declaración tiene dos tipos de elementos, el primero es la cadena de caracteres o texto (char *formato) que se va a imprimir por pantalla; y el segundo elemento contiene especificadores de formato, que definen la forma en que se muestran los argumentos.

1.3.4. Especificadores de Formato

El formato de las cadenas de texto contiene símbolos de formateo de distintos tipos de datos. Estos símbolos son reemplazados por valores almacenados en variables, como se muestra en el programa 1.13. Los principales especificadores de formato que maneja el lenguaje C/C++ son los que se muestran en la Tabla 2.1.

Tabla 2.1. Especificadores de Formato.

Código	Descripción
%c	Formato de carácter a cadena de caracteres (string)
%d	Formato de entero a cadena de caracteres
%ld	Formato de entero largo a cadena de caracteres
%f	Formato de punto flotante a cadena de caracteres
%lf	Formato de punto flotante de doble precisión a cadena de caracteres
%s	Formato de cadena de caracteres a cadena de caracteres
%p	Formato de puntero a cadena de caracteres

1.3.5. Secuencias de Escape

Existen caracteres especiales, llamados secuencias de escape o caracteres de escape. Una secuencia de escape está simbolizada con un backslash '\' seguido por un caracter(es) común. Por lo tanto, en el programa 1.8, el caracter de nueva línea está simbolizado por '\n'. La Tabla 2.2 muestra las secuencias de escape más utilizadas:

Tabla 2.2. Secuencias de Escape.

Código	Descripción
\n	Caracter de nueva línea: Representa a una nueva línea (intro).
\t	Caracter de tabulación: Representa a una tabulación (tab space).
\a	Caracter de alerta: Representa a una alerta.
\\	Backslash: Representa a un caracter de backslash.
\'	Símbolo de comilla simple.
\"	Símbolo de comillas dobles.

Problema 1.8: Escribir un programa que permita demostrar el uso de secuencias de escape.

Programa 1.8: Código del programa.

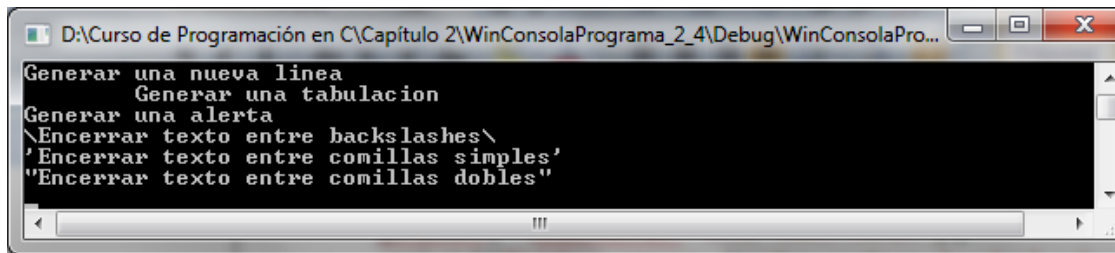
```
/* *****  
WinConsolaPrograma_1_8  
***** */  
  
// Librerías.  
#include <stdio.h>  
#include <stdlib.h>  
  
// Función principal.
```



```
int main()
{
    // Utilización de secuencias de escape.
    printf("Generar una nueva linea\n");
    printf("\tGenerar una tabulacion\n");
    printf("\aGenerar una alerta\n");
    printf("\\Encerrar texto entre backslashes\\n");
    printf("'Encerrar texto entre comillas simples'\n");
    printf("\"Encerrar texto entre comillas dobles\"n");

    // Incorporar una pausa en el programa.
    system("pause");
    return 0;
}
```

Salida 1.8: Salida del programa.



1.3.6. Función de Entrada scanf()

La función `scanf()`, es una función de entrada de datos por consola, que lee todos los tipos de datos definidos en C/C++, convirtiendo automáticamente al formato interno apropiado de acuerdo a los especificadores de formato. El prototipo o la declaración de la función '`scanf()`' es el siguiente:

```
int scanf(const char *formato [, direcciones, ...]);
```

La función '`scanf()`', como se puede ver en la declaración tiene dos tipos de elementos, el primero es la cadena de caracteres o texto (`char *formato`) que se va a leer por pantalla con su respectivo especificador de formato; y el segundo elemento contiene las direcciones de memoria de las variables que se van a leer.

Problema 1.9: Escribir un programa que permita preguntarle al usuario su primer nombre y donde luego la computadora responda Hola usuario XYZ.

Programa 1.9: Código del programa.

```
/* *****
WinConsolaPrograma_1_9
***** */

// Librerías.
#include <stdio.h>
#include <stdlib.h>

// Función principal.
int main()
{
    // Declaración de una variable.
    char letra;

    // Leer una letra.
```

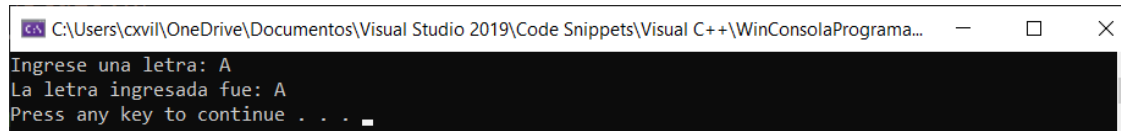
Lenguaje C/C++. Problemas, Casos y Proyectos

```
printf("Ingrese una letra: ");
scanf_s("%c", &letra);

// Imprimir el mensaje de "La letra ingresada fue:" y luego
// imprimir un salto de línea con la secuencia de escape "\n".
printf("La letra ingresada fue: %c\n", letra);

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.9: Salida del programa.



Como se puede ver en este programa, nosotros desplegamos el texto: “Ingrese una letra: ” en la consola con la siguiente instrucción:

```
printf("Ingrese una letra: ");
```

Luego nosotros le pedimos al usuario que ingrese una letra con la siguiente instrucción:

```
scanf_s("%c", &letra);
```

Finalmente nosotros desplegamos el mensaje “La letra ingresada fue: ” seguido por el especificador de formato “%c” para imprimir el valor de una variable de tipo char (carácter) y luego se imprime un salto de línea con la secuencia de escape “\n”, con la siguiente instrucción:

```
printf("La letra ingresada fue: %c\n", letra);
```

Tip de Programación: Solo en la herramienta de programación Visual C++ de Microsoft se utiliza la función `scanf_s()`, mientras que en otras herramientas de programación como DEV-C++, Code::Blocks, GDB online Debugger, entre otras, se utiliza directamente la función `scanf()`.

1.3.7. Funciones de Utilidades Generales Estándar

El archivo de la librería de utilidades estándar “`cstdlib`” contiene un sinnúmero de funciones para gestionar la memoria dinámica, para la comunicación con el ambiente, entre otras. Una de las funciones más importantes de esta librería es la función `system` que permite invocar al procesador de comandos para ejecutar un comando y la función `system(“pause”)` permite incorporar una pausa en el programa.

1.4. Variables

En el programa 1.7, le pedimos al usuario que ingrese su nombre. Luego el programa dice “Hola, ” a aquel usuario, escribiendo su nombre. La computadora sabe a qué nombre debe decirle “Hola, ” debido a que nosotros guardamos el nombre del usuario con la siguiente instrucción:

```
cin >> primerNombre;
```

Lenguaje C/C++. Problemas, Casos y Proyectos

El comando `cin >>` espera que el usuario ingrese un texto que corresponde a su nombre y luego el programa guarda en una variable de tipo `string` llamada `primerNombre`. Debido a que el nombre del usuario fue guardado en una variable, nosotros podemos desplegar el valor de la variable que contiene un nombre con la siguiente línea de código:

```
cout << "Hola, " << primerNombre << endl;
```

En el programa 1.9, le pedimos al usuario que ingrese una letra. Luego el programa dice “La letra ingresada fue: ” al usuario, escribiendo una letra. La computadora sabe qué letra ingresó el usuario, debido a que nosotros guardamos la letra en una variable con la siguiente instrucción:

```
scanf_s("%c", &letra);
```

El comando `scanf` espera que el usuario ingrese una letra y luego el programa guarda un valor en una variable de tipo `char` llamada `letra`. Debido a que la letra que ingresó el usuario fue guardada en una variable, nosotros podemos desplegar el valor de la variable que contiene la letra con la siguiente línea de código:

```
printf("La letra ingresada fue: %c\n", letra);
```

1.4.1. Definición de Variable

Una variable se define como un elemento que ocupa una región física del sistema de memoria de acceso aleatorio (RAM) y almacena un valor de algún tipo (Ceballos, F.J., 2007). En el lenguaje C/C++ se debe declarar las variables previamente antes de usarlas, porque en la compilación del programa se asigna el espacio de memoria a las variables de acuerdo a su tipo de dato. La dirección de memoria donde se ubica una variable se representa en formato hexadecimal (hex). Por cuestiones didácticas vamos a utilizar valores en formato decimal (dec) para representar las direcciones de memoria de las variables. En la Figura 1.16 se muestra un ejemplo de una variable llamada ‘c’ de tipo entero, cuyo valor almacenado es el número ‘7’ en la dirección de memoria hex 1000.

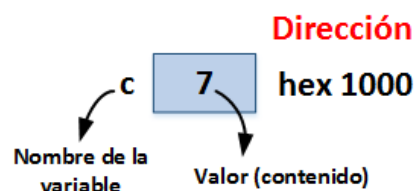


Figura 1.16. Representación gráfica de la variable ‘c’.

1.4.2. Tipos de Datos en C/C++

El tipo de dato define un conjunto de valores que puede tener una variable, junto con un conjunto de operaciones que se pueden realizar sobre esa variable (Granizo, E., 2016). Los tipos de datos comunes son los enteros, números reales y caracteres alfanuméricos. La Tabla 1.2 resume todos los tipos de datos que soporta el lenguaje C/C++.

Tabla 1.2. Tipos de Datos.

Tipo de Dato	Descripción	Rango	Tamaño en Bytes
char	Utilizado para almacenar un simple carácter tales como: ‘a’, ‘b’, ‘c’, etc.	[-128, 127]	1
short	Utilizado para almacenar valores enteros pequeños.	[-32768, 32767]	2

Lenguaje C/C++. Problemas, Casos y Proyectos

int	Utilizado para almacenar valores enteros.	[-2147483648, 2147483647]	4
long	Utilizado para almacenar valores enteros grandes.	[-2147483648, 2147483647]	4
float	Utilizado para almacenar valores de números con coma flotante.	$\pm[1.2 \times 10^{-38}, 3.4 \times 10^{38}]$	4
double	Utilizado para almacenar valores de números con coma flotante grandes o de doble precisión.	$\pm [2.2 \times 10^{-308}, 1.8 \times 10^{308}]$	8
bool	Utilizado para almacenar valores de verdad que pueden ser verdadero (true) y falso (false). Cabe notar que el valor de cero es considerado falso y cualquier otro valor que no se cero es considerado verdadero.	[0, 1]	1
string	Utilizado para almacenar variables de tipo de cadenas de caracteres. Cabe notar que para utilizar este tipo de dato se debe utilizar el espacio de nombre 'std'.	[0, 2000 millones de caracteres]	1 Byte por caracter

El tamaño en bytes de los tipos de datos, dependen de la plataforma. Por ejemplo, si usted asume en su código que un char ocupa 1 Byte en una plataforma de 16-bits, y luego se mueve a una plataforma de desarrollo de 32-bits, puede ser que haya ajustes en los tamaños de los datos.

1.4.3. Declaración y Definición de Variables

Una variable se declara de acuerdo con la siguiente sintaxis:

Sintaxis:

Tipo_de_Dato Nombre_de_la_Variable;

Tipo_de_Dato

Tipo de dato definido por el lenguaje C/C++.

Nombre_de_la_Variable

Identificador de la variable.

Por ejemplo:

```
char letra;
int num_entero;
float num_flotante;
```

Conforme a este ejemplo se puede ver que se han declarado tres variables, cada una con un nombre específico y con un tipo de dato. Sin embargo, estas variables que han sido declaradas no están definidas, es decir, el valor de almacenamiento de cada una de ellas es desconocido. Consecuentemente, es común decir que estas variables contienen basura (garbage).

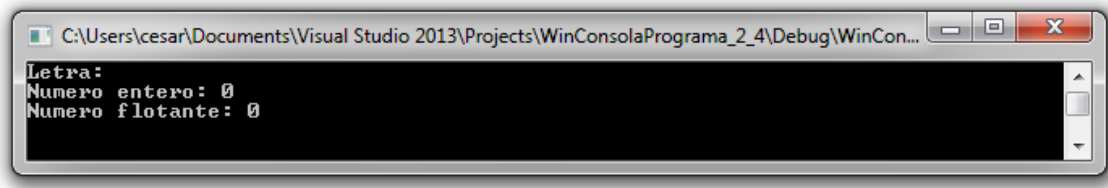
Problema 1.10: Escribir un programa que permita declarar tres variables, una de tipo caracter, otra de tipo entero y otra de tipo flotante; y luego se imprima el valor asignado por defecto a esas variables, utilizando las funciones 'cout' y 'cin'.

Programa 1.10: Código del programa.

```
/******
WinConsolaPrograma_1_10
```

```
*****/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // Declaración y definición de variables.  
    char letra = ' ';  
    int num_entero = 0;  
    float num_flotante = 0.0f;  
  
    // Imprimir los valores asignados por defecto de las  
    // variables declaradas.  
    cout << "Letra: " << letra << endl;  
    cout << "Numero entero: " << num_entero << endl;  
    cout << "Numero flotante: " << num_flotante << endl;  
  
    // Incorporar una pausa en el programa.  
    system("pause");  
    return 0;  
}
```

Salida 1.10: Salida del programa.



En el programa 1.10 se utiliza la función ‘cout’ que como se indicó en la sección 1.3.1, sirve para imprimir mensajes de texto y datos (salida por consola). En este caso para imprimir los contenidos de las variables que tienen valores por defecto, después de haber sido declaradas e inicializadas, se utiliza el operador de inserción ‘<<’ para imprimir tanto mensajes de texto como para imprimir los valores de las variables.

Problema 1.11: Escribir un programa que permita declarar tres variables, una de tipo caracter, otra de tipo entero y otra de tipo flotante; y luego se imprima el valor asignado por defecto a esas variables, utilizando las funciones printf y scanf.

Programa 1.11: Código del programa.

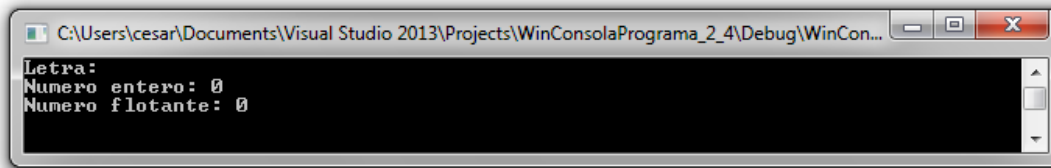
```
/*  
*****/  
WinConsolaPrograma_1_11  
*****/  
  
// Librerías.  
#include <stdio.h>  
#include <cstdlib>  
  
// Función principal.  
int main()  
{  
    // Declaración y definición de variables.  
    char letra = ' ';  
    int num_entero = 0;
```

```
float num_flotante = 0.0f;

// Imprimir los valores asignados por defecto de las
// variables declaradas.
printf("Letra: %c \n", letra);
printf("Numero entero: %d \n", num_entero);
printf("Numero flotante: %f \n", num_flotante);

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.11: Salida del programa.



En el programa 1.11 se utiliza la función `printf()` que como se indicó en la sección 1.3.3, sirve para imprimir mensajes de texto y datos (salida por consola). En este caso para imprimir los contenidos de las variables que tienen valores por defecto, después de haber sido declaradas e inicializadas, se utilizan especificadores de formato, para imprimir cada tipo de dato (char, int, float), que como se muestra en la Figura 1.17, el especificador de formato correspondiente a un tipo de dato específico, se reemplaza por el valor almacenado en una variable.

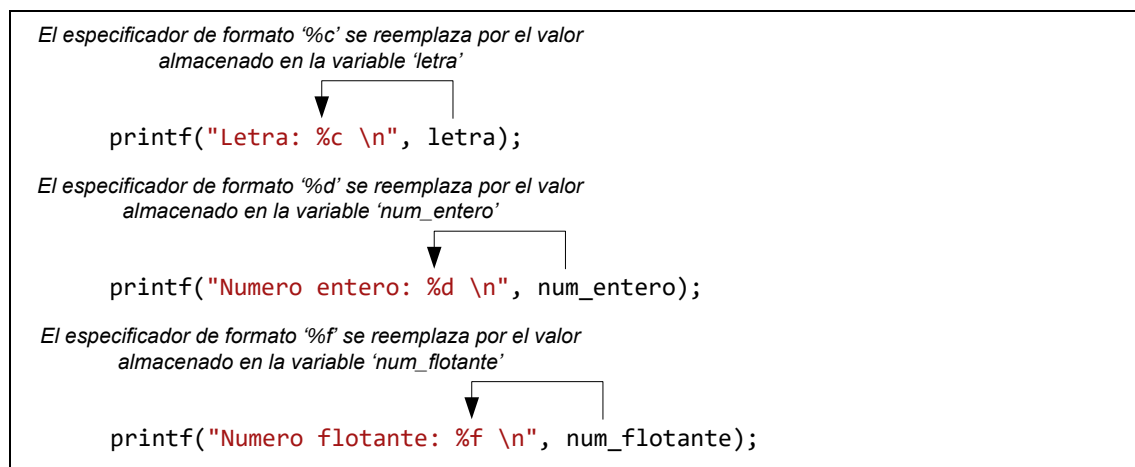


Figura 1.17. Utilización de especificadores de formato en la función `printf()`.

		Dirección
letra	'M'	hex 2000
		Dirección
num_entero	20	hex 3000
		Dirección
num_flotante	17.23	hex 4000

Figura 1.18. Representación gráfica de tres variables.

Como se puede ver en la salida del programa 1.11, se imprimen valores por defecto y no basura (garbage), ya que las variables han sido declaradas y luego inicializadas.

Cuando se asigna un valor a una variable, se dice que estamos definiendo o inicializando la variable. Por ejemplo:

```
letra = 'M';  
num_entero = 20;  
num_flotante = 17.23;
```

La representación gráfica de cada una de estas tres variables en la memoria RAM se muestra en la Figura 1.18, como son: a) variable 'letra' de tipo 'char', cuyo valor almacenado es el carácter 'M' en la dirección de memoria hex 2000; b) variable 'num_entero' de tipo 'int', cuyo valor almacenado es el número '20' en la dirección de memoria hex 3000; c) variable 'num_flotante' de tipo 'float', cuyo valor almacenado es el número real '17.23' en la dirección de memoria hex 4000.

1.4.4. Operador de Asignación Igual (=)

El símbolo (=), en C/C++, se conoce como el **operador de asignación**, que permite asignar valores a las variables y a fórmulas matemáticas (Joyanes, L., Zahonero, I., 2005). Este operador no tiene nada que ver con el operador de igualdad que se utiliza para comparación, donde el símbolo que se ha definido en C/C++ para la comparación es el (==).

Se puede declarar y definir una variable al mismo tiempo en C/C++. Por ejemplo:

```
char letra = 'M';  
int num_entero = 20;  
float num_flotante = 17.23;
```

Debido a que en la declaración de las variables se almacena basura, se recomienda siempre definir o inicializar las variables después de la declaración con algún valor por defecto. Generalmente, se utiliza el valor de cero para inicializar valores numéricos y un carácter en blanco (' ') para valores de caracteres.

Se puede hacer múltiples declaraciones y/o definiciones en una sola instrucción utilizando el operador coma (,). Por ejemplo:

```
int x, y, z;  
x = 1, y = 3, z = -2;  
float a = 2.1f, b = 3.5f, c = 4.9f;
```

Se recomienda hacer una declaración o definición por línea, para detectar más fácilmente errores.

Finalmente se puede también enlazar asignaciones juntas. El flujo de los valores asignados va desde la derecha hacia la izquierda. Por ejemplo:

```
float n = 1.0f;  
float x, y, z;  
x = y = z = n;
```

Problema 1.12: Escribir un programa que permita leer e imprimir un dato de tipo carácter, un dato de tipo entero y un dato de tipo flotante, declarando y definiendo las variables previamente, utilizando las funciones 'cout' y 'cin'.

Programa 1.12: Código del programa.

```
/*  
WinConsolaPrograma_1_12  
*/
```



```
*****/

// Librerías.
#include <iostream>
#include <cstdlib>

using namespace std;

// Función principal.
int main()
{
    // Declaración e inicialización de variables.
    char letra = ' ';
    int num_entero = 0;
    float num_flotante = 0.0;

    // Imprimir un mensaje y leer un dato.
    cout << "Ingrese una letra: "; // Salida.
    cin >> letra;                  // Entrada.

    // Imprimir un mensaje y leer un dato.
    cout << "Ingrese un numero entero: "; // Salida.
    cin >> num_entero;                  // Entrada.

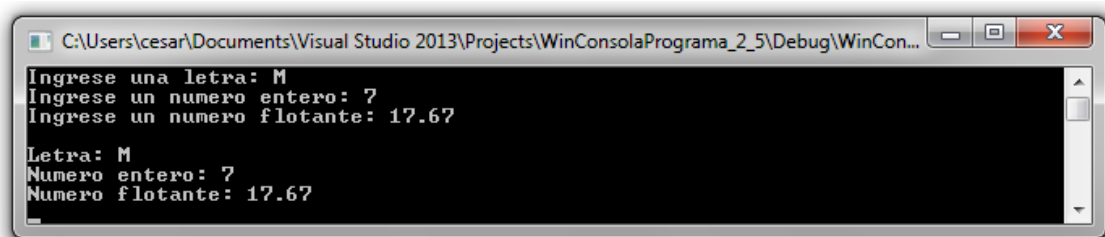
    // Imprimir un mensaje y leer un dato.
    cout << "Ingrese un numero flotante: "; // Salida.
    cin >> num_flotante; // Entrada.

    // Imprimir un caracter de nueva línea.
    cout << endl;

    // Imprimir los valores que el usuario ha ingresado.
    cout << "Letra: " << letra << endl;
    cout << "Numero entero: " << num_entero << endl;
    cout << "Numero flotante: " << num_flotante << endl;

    // Incorporar una pausa en el programa.
    system("pause");
    return 0;
}
```

Salida 1.12: Salida del programa.



Primeramente, en el programa 1.12, se declaran y se definen tres variables: a) letra, de tipo caracter (char), inicializada con el valor de un espacio en blanco (' '); b) num_entero, de tipo entero (int), inicializada con el valor de cero (0); c) num_flotante, de tipo flotante (float), inicializada con el valor de cero (0.0f).

Luego, en este programa se utiliza la función 'cin >>' que como se indicó en la sección 1.3.2, sirve para leer datos (entrada por consola). En este caso se leen tres datos, uno de tipo char, otro de tipo int y otro de tipo float, cuyas variables van a ser modificadas en su contenido con valores que el usuario ingresa por teclado a la consola.

Lenguaje C/C++. Problemas, Casos y Proyectos

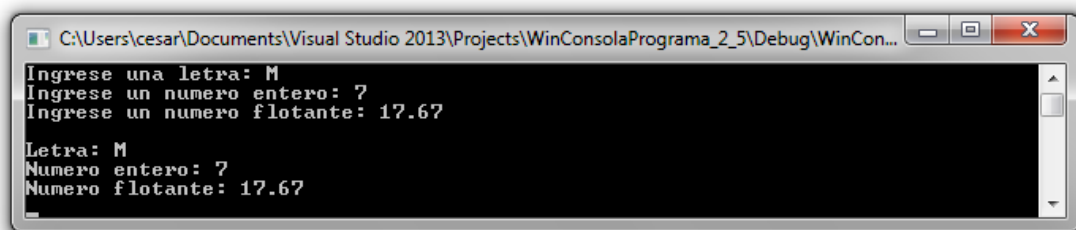
Finalmente, se utiliza la función 'cout <<' que como se indicó en la sección 1.3.1, sirve para imprimir mensajes de texto y datos (salida por consola). En este caso, esta función se utilizó para imprimir los tres datos leídos por teclado.

Problema 1.13: Escribir un programa que permita leer e imprimir un dato de tipo caracter, un dato de tipo entero y un dato de tipo flotante, declarando y definiendo las variables previamente, utilizando las funciones 'printf' y 'scanf'.

Programa 1.13: Código del programa.

```
/* *****  
WinConsolaPrograma_1_13  
***** */  
  
// Librerías.  
#include <stdio.h>  
#include <stdlib.h>  
  
// Función principal.  
int main()  
{  
    // Declaración de variables.  
    char letra = ' ';  
    int num_entero = 0;  
    float num_flotante = 0.0f;  
  
    // Imprimir un mensaje y leer un dato.  
    printf("Ingrese una letra: "); // Salida.  
    scanf_s("%c", &letra);        // Entrada.  
  
    // Imprimir un mensaje y leer un dato.  
    printf("Ingrese un numero entero: "); // Salida.  
    scanf_s("%d", &num_entero);        // Entrada.  
  
    // Imprimir un mensaje y leer un dato.  
    printf("Ingrese un numero flotante: "); // Salida.  
    scanf_s("%f", &num_flotante);      // Entrada.  
  
    // Imprimir un caracter de nueva línea.  
    printf("\n"); // Secuencia de escape.  
  
    // Imprimir los valores que el usuario ha ingresado.  
    printf("Letra ingresada: %c \n", letra);  
    printf("Numero entero ingresado: %d \n", num_entero);  
    printf("Numero flotante ingresado: %f \n", num_flotante);  
  
    // Incorporar una pausa en el programa.  
    system("pause");  
    return 0;  
}
```

Salida 1.13: Salida del programa.



Lenguaje C/C++. Problemas, Casos y Proyectos

Primeramente, en el programa 1.13, se declaran y se definen tres variables: a) letra, de tipo carácter (char), inicializada con el valor de un espacio en blanco (' '); b) num_entero, de tipo entero (int), inicializada con el valor de cero (0); c) num_flotante, de tipo flotante (float), inicializada con el valor de cero (0.0f).

Luego, en este programa se utiliza la función scanf() que como se indicó en la sección 1.3.6, sirve para leer datos (entrada por consola). En este caso se leen tres datos, uno de tipo char, otro de tipo int y otro de tipo float, donde los argumentos que se envían a la función son: a) el especificador de formato, que indica que la variable que se envía corresponde a un tipo de dato específico, por ejemplo, char, int, float, etc.; b) la dirección de la variable que va a ser modificada en su contenido, con un valor que el usuario ingresa por teclado a la consola; como se muestra en la Figura 1.19.

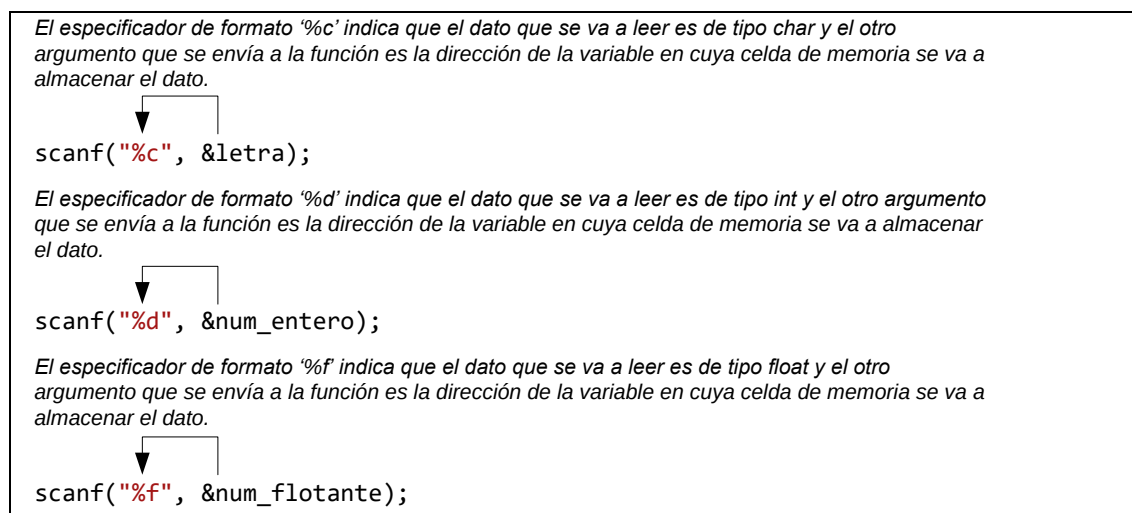


Figura 1.19. Utilización de especificadores de formato en la función scanf().

Finalmente, en este programa se utiliza la función printf() que como se indicó en la sección 1.3.3, sirve para imprimir mensajes de texto y datos (salida por consola). En este caso para imprimir los tres datos leídos, se utilizan especificadores de formato, para imprimir cada tipo de dato (char, int, float), que como se muestra en Figura 1.20, el especificador de formato correspondiente a un tipo de dato específico, se reemplaza por el valor almacenado en una variable.

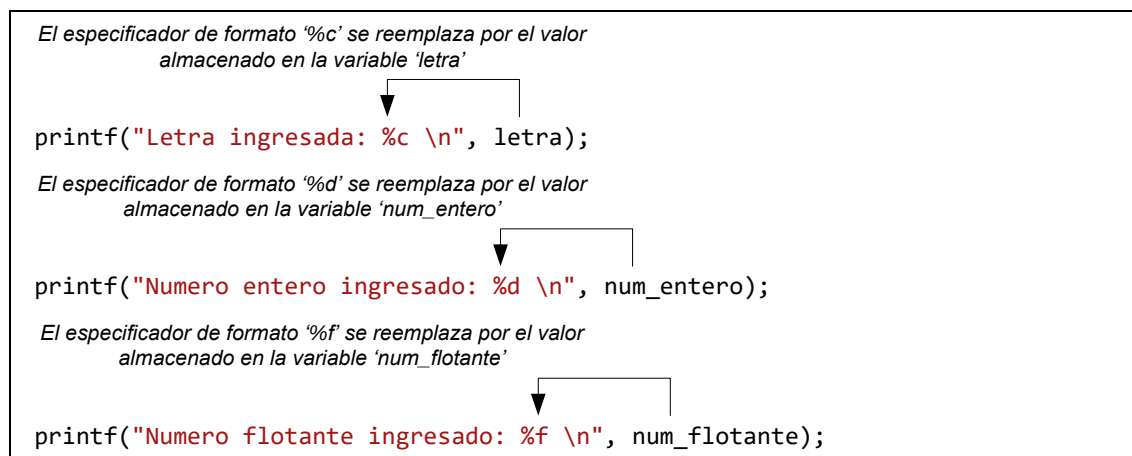


Figura 1.20. Utilización de especificadores de formato en la función printf().

1.4.5. Nombres de Variables

Como se mostró en el programa 1.10, cuando se declaran y/o se definen variables, nosotros debemos ponerles un nombre a éstas (identificador), para podernos referir a cada una de las

Lenguaje C/C++. Problemas, Casos y Proyectos

variables dentro del programa (Jones, B., Aitken, P., 2003). Los nombres de las variables pueden tener cualquier cosa, con algunas excepciones, como se mencionan a continuación:

1. Los nombres de las variables deben comenzar con una letra. El nombre de la variable 5MiVariable es ilegal. Sin embargo, el caracter guión bajo (underscore) se lo considera una letra, y por lo tanto, el identificador `_MiVariable` es legal.
2. Los nombres de las variables pueden incluir el caracter guión bajo o underscore ('_'), letras, y números, pero no símbolos. Por lo que, usted no puede utilizar símbolos como: '!', '"', '#', '\$', '%', '&', '\', etc., en los nombres de las variables.
3. Los nombres de las variables no pueden ser palabras reservadas de C/C++, como: `char`, `int`, `float`, `cout`, `cin`, `_getch`, etc. Por ejemplo, usted no puede nombrarle a una variable como "float", ya que es una palabra reservada de C que especifica el tipo de dato float.
4. Los nombres de las variables no pueden tener espacios en blanco entre ellas. Por ejemplo, el nombre de la variable "`_ M i _ Variable`" es ilegal, lo correcto sería tener el siguiente nombre: "`_Mi_Variable`".

Tip de Programación: El lenguaje C/C++ es case sensitive (letra sensitiva). Por ejemplo, las siguientes palabras: `hola`, `Hola`, `HOLA`, `hola`, aunque quieren decir lo mismo, es decir, son únicas, ellas difieren por la letra (mayúscula y minúscula).

1.5. Operadores Aritméticos

Adicionalmente a la declaración, definición, lectura e impresión de variables de diferentes tipos de datos que maneja el lenguaje C/C++, nosotros podemos ejecutar operaciones aritméticas básicas entre ellas.

1.5.1. Operadores Aritméticos Binarios

El lenguaje C/C++ maneja cinco operadores aritméticos binarios:

1. Operador Suma (+).
2. Operador Resta (-).
3. Operador de Multiplicación (*).
4. Operador de División (/).
5. Operador de Módulo (%).



Figura 1.21. Operadores Aritméticos Binarios.

Las operaciones de suma, resta, multiplicación y división están definidas para todos los tipos de datos numéricos (ver Figura 1.21). El operador de módulo está definido sólo para operaciones con enteros (Davies, P., 1995). El programa 1.14 muestra el uso de estos operadores.

Problema 1.14: Escribir un programa que permita presentar el uso de los operadores aritméticos binarios con los tipos de datos enteros de C/C++, utilizando las funciones 'cout' y 'cin'.

Programa 1.14: Código del programa.

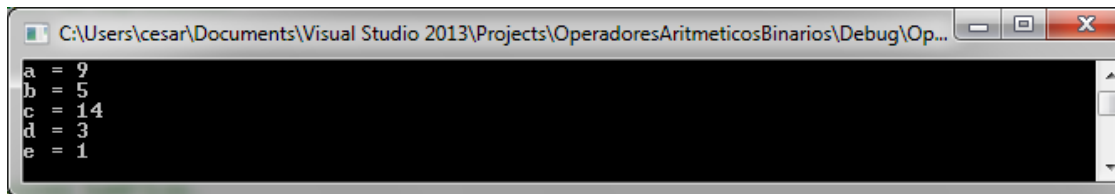
```
/******  
WinConsolaPrograma_1_14  
*****/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
  
using namespace std;  
  
// Función principal.
```

```
int main()
{
    // Declaración de variables y definición de expresiones.
    int a = 7 + 2;
    int b = 7 - 2;
    int c = 7 * 2;
    int d = 7 / 2;
    int e = 7 % 2;

    // Imprimir los valores de las expresiones numéricas.
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    cout << "c = " << c << endl;
    cout << "d = " << d << endl;
    cout << "e = " << e << endl;

    // Incorporar una pausa en el programa.
    system("pause");
    return 0;
}
```

Salida 1.14: Salida del programa.



Problema 1.15: Escribir un programa que permita presentar el uso de los operadores aritméticos binarios con los tipos de datos enteros de C/C++, utilizando las funciones 'printf' y 'scanf'.

Programa 1.15: Código del programa.

```
/* *****
WinConsolaPrograma_1_15
***** */

// Librerías.
#include <stdio.h>
#include <cstdlib>

// Función principal.
int main()
{
    // Declaración de variables y definición de expresiones.
    int a = 7 + 2;
    int b = 7 - 2;
    int c = 7 * 2;
    int d = 7 / 2;
    int e = 7 % 2;

    // Imprimir los valores de las expresiones numéricas.
    printf("a = %d\n", a);
    printf("b = %d\n", b);
    printf("c = %d\n", c);
    printf("d = %d\n", d);
    printf("e = %d\n", e);

    // Incorporar una pausa en el programa.
    system("pause");
}
```

```
    return 0;
}
```

Salida 1.15: Salida del programa.

```
C:\Users\cesar\Documents\Visual Studio 2013\Projects\OperadoresAritmeticosBinarios\Debug\Op...
a = 9
b = 5
c = 14
d = 3
e = 1
```

1.5.2. El Operador de Módulo

El lenguaje C/C++ maneja un operador especial que es el de módulo, que retorna el resto de una división entera. Por ejemplo, al dividir la fracción $7 / 2$, el resto de la división es 1; al dividir la fracción $7 / 13$, el resto de la división es 7; y al dividir la fracción $23 / 8$, el resto de esta división es 7 que a su vez es el numerador de la fracción $7 / 8$ que forma parte de la fracción impropia $2\frac{7}{8}$, como se muestra a continuación:

$$\frac{7}{2} = 1 \frac{2}{3} \quad \frac{7}{13} = 0 \frac{7}{13} \quad \frac{23}{8} = 2 \frac{7}{8}$$

1.5.3. Expresiones Aritméticas Compuestas

El lenguaje C/C++ define las siguientes expresiones aritméticas compuestas que utilizan un operador aritmético y el operador de asignación junto a dos operandos (Davies, P., 1995). La Tabla 1.3 resume lo expuesto:

Tabla 1.3. Expresiones Aritméticas Compuestas.

Expresión Aritmética Compuesta	Descripción	Ejemplo
$x = x + y;$	Asigna el valor de $x + y$ en la misma variable x .	<pre>int x = 5; int y = 7; x = x + y; // x = 12</pre>
$x = x - y;$	Asigna el valor de $x - y$ en la misma variable x .	<pre>int x = 5; int y = 7; x = x - y; // x = -2</pre>
$x = x * y;$	Asigna el valor de $x * y$ en la misma variable x .	<pre>int x = 5; int y = 7; x = x * y; // x = 35</pre>
$x = x / y;$	Asigna el valor de x / y en la misma variable x .	<pre>int x = 5; int y = 7; x = x / y; // x = 0</pre>
$x = x \% y;$	Asigna el valor de $x \% y$ en la misma variable x .	<pre>int x = 5; int y = 7; x = x \% y; // x = 5</pre>

Problema 1.16: Escribir un programa que permita probar el uso de las cinco expresiones aritméticas compuestas, con dos números enteros leídos desde el teclado, utilizando las funciones 'cout' y 'cin'.

Programa 1.16: Código del programa.

```
/*
WinConsolaPrograma_1_16
*/
```

```
// Librerías.
#include <iostream>
#include <cstdlib>

using namespace std;

// Función principal.
int main()
{
    // declaración de variables.
    int x = 0, y = 0;

    // Imprimir un mensaje de información.
    cout << "Uso de las operaciones aritméticas compuestas." << endl;
    cout << endl;

    // Imprimir un mensaje y leer un dato de tipo entero.
    cout << "Ingrese el primer número: "; // Salida.
    cin >> x;                               // Entrada.

    // Imprimir un mensaje y leer un dato de tipo entero.
    cout << "Ingrese el segundo número: "; // Salida.
    cin >> y;                               // Entrada.

    // Almacenar en variables separadas el valor de 'x', de tal
    // manera que cada variable sea independiente entre sí.
    int a = x;
    int b = x;
    int c = x;
    int d = x;
    int e = x;

    // Utilizar las expresiones aritméticas compuestas.
    a = a + y;
    b = b - y;
    c = c * y;
    d = d / y;
    e = e % y;

    // Imprimir los resultados.
    cout << endl;
    cout << "x = x + y = " << a << endl;
    cout << "x = x - y = " << b << endl;
    cout << "x = x * y = " << c << endl;
    cout << "x = x / y = " << d << endl;
    cout << "x = x % y = " << e << endl;

    // Incorporar una pausa en el programa.
    system("pause");
    return 0;
}
```


Salida 1.16: Salida del programa.

```
C:\Users\cxvil\OneDrive\Documentos\Visual Studio 2019\Code Snippets\Visual C++\WinConsola...
Uso de las operaciones aritméticas compuestas.

Ingrese el primer n-mero: 5
Ingrese el segundo n-mero: 7

x = x + y = 12
x = x - y = -2
x = x * y = 35
x = x / y = 0
x = x y = 5
Press any key to continue . . .
```

Problema 1.17: Escribir un programa que permita probar el uso de las cinco expresiones aritméticas compuestas, con dos números enteros leídos desde el teclado, utilizando las funciones 'printf' y 'scanf'.

Programa 1.17: Código del programa.

```
/* *****
WinConsolaPrograma_1_17
***** */

// Librerías.
#include <stdio.h>
#include <stdlib.h>

// Función principal.
int main()
{
    // Declaración de variables.
    int x = 0, y = 0;

    // Imprimir un mensaje de información.
    printf("Uso de las operaciones aritméticas compuestas.\n\n");

    // Imprimir un mensaje y leer un dato de tipo entero.
    printf("Ingrese el primer número: "); // Salida.
    scanf_s("%d", &x); // Entrada.

    // Imprimir un mensaje y leer un dato de tipo entero.
    printf("Ingrese el segundo número: "); // Salida.
    scanf_s("%d", &y); // Entrada.

    // Almacenar en variables separadas el valor de 'x', de tal
    // manera que cada variable sea independiente entre sí.
    int a = x;
    int b = x;
    int c = x;
    int d = x;
    int e = x;

    // Utilizar las expresiones aritméticas compuestas.
    a = a + y;
    b = b - y;
    c = c * y;
    d = d / y;
    e = e % y;

    // Imprimir los resultados.
    printf("\n");
}
```

```
printf("x = x + y = %d\n", a);
printf("x = x - y = %d\n", b);
printf("x = x * y = %d\n", c);
printf("x = x / y = %d\n", d);
printf("x = x % y = %d\n", e);

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.17: Salida del programa.

1.5.4. Operadores Aritméticos Unarios

Un operador unario es una operación que actúa sobre una variable u operando (Davies, P., 1995). La Tabla 1.4 resume los tres operadores unarios que maneja el C/C++.

Tabla 1.4. Operadores Unarios.

Operador Unario	Descripción	Ejemplo
Operador de Negación: -	Niega el operando.	<code>int a = 5;</code> <code>int b = -a; // b = -5</code>
Operador de Incremento: ++	Incrementa el valor del operando en uno. Este operador puede tener una notación prefija o postfija.	<code>int x = 5;</code> <code>++x; // x = 6 (prefijo)</code> <code>x++; // x = 7 (postfijo)</code>
Operador de Decremento: --	Decrementa el valor del operando en uno. Este operador puede tener una notación prefija o postfija.	<code>int y = 5;</code> <code>--y; // y = 4 (prefijo)</code> <code>y--; // y = 3 (postfijo)</code>

Tip de Programación: El operador de incremento ++ utilizado en notación postfija del ejemplo anterior es equivalente a: $x = x + 1$; mientras que el operador de decremento--utilizado en notación postfija del ejemplo anterior es equivalente a: $y = y - 1$.

En este curso de programación se recomienda el uso de los operadores ++ y -- en notación postfija, debido a la complejidad de comprensión que implica la notación prefija.

Observe que los operadores de incremento y decremento pueden tener una notación prefija y postfija, que aparentemente da lo mismo utilizar, pero no es así. La técnica que diferencia el uso de esta notación en expresiones simples y complejas es la siguiente:

1. `a++`; significa que primero se utiliza el valor de la variable y luego se incrementa el valor de la variable en uno.
2. `++a`; significa que primero se incrementa el valor de la variable en uno y luego se utiliza el valor de la variable.

3. `a--`; significa que primero se utiliza el valor de la variable y luego se decrementa el valor de la variable en uno.
4. `--a`; significa que primero se decrementa el valor de la variable en uno y luego se utiliza el valor de la variable.

Los siguientes programas muestran el uso de estos operadores.

Problema 1.18: Escribir un programa que permita probar el uso de los operadores aritméticos unarios en expresiones matemáticas simples, utilizando las funciones 'cout' y 'cin'.

Programa 1.18: Código del programa.

```
/******  
WinConsolaPrograma_1_18  
******/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // Uso de los operadores aritméticos unarios en expresiones simples.  
    cout << "Uso de los operadores aritméticos unarios";  
    cout << " en expresiones simples." << endl;  
    cout << endl;  
  
    // Operador de negación.  
    cout << "Uso del operador de negación." << endl;  
    int a = 5;  
    cout << "El valor de a = " << a << endl;  
    int b = -a; // b = -5  
    cout << "El valor de b = " << b << endl;  
    cout << endl;  
  
    // Operador de incremento.  
    cout << "Uso del operador de incremento." << endl;  
    int x = 5;  
    cout << "El valor de x = " << x << endl;  
    ++x; // x = 6 (prefijo)  
    cout << "El valor de ++x = " << x << endl;  
    x++; // x = 7 (postfijo)  
    cout << "El valor de x++ = " << x << endl;  
    cout << endl;  
  
    // Operador de decremento.  
    cout << "Uso del operador de decremento." << endl;  
    int y = 5;  
    cout << "El valor de y = " << y << endl;  
    --y; // y = 4 (prefijo)  
    cout << "El valor de --y = " << y << endl;  
    y--; // y = 3 (postfijo)  
    cout << "El valor de y-- = " << y << endl;  
    cout << endl;  
  
    // Incorporar una pausa en el programa.  
    system("pause");  
    return 0;  
}
```

Salida 1.18: Salida del programa.

```
C:\Users\cxvil\OneDrive\Documentos\Visual Studio 2019\Code Snippets\Visual C++\WinConsolaPro...
Uso de los operadores aritméticos unarios en expresiones simples.

Uso del operador de negación.
El valor de a = 5
El valor de b = -5

Uso del operador de incremento.
El valor de x = 5
El valor de ++x = 6
El valor de x++ = 7

Uso del operador de decremento.
El valor de y = 5
El valor de --y = 4
El valor de y-- = 3

Press any key to continue . . .
```

Tabla 1.18: Prueba de escritorio manual del programa.

Nº Paso	a	b	x	y
1	5			
2		-5		
3			5	
4			6	
5			7	
6				5
7				4
8				3

1.5.5. Prueba de Escritorio

La prueba de escritorio es una técnica que permite controlar el flujo de los datos y ver los valores que van tomando las variables de un programa en el momento de la ejecución del mismo. En la Tabla 1.18 que forma parte del Problema 1.18 se presenta una prueba de escritorio manual donde en la primera columna está cada uno de los pasos que se cumplen desde el inicio del programa hasta la finalización del mismo y en las otras columnas los valores que van tomando cada una de las variables en cada uno de los pasos del programa.

Se puede realizar también la prueba de escritorio con la computadora, para lo cual se utiliza una de las herramientas de depuración (Depurar) de Visual C++ 2021, la cual la encontramos en la barra de menús y la activamos con la tecla F10 que permite hacer una prueba de escritorio paso a paso por procedimientos, como se muestra en la Figura 1.22.

Lenguaje C/C++. Problemas, Casos y Proyectos

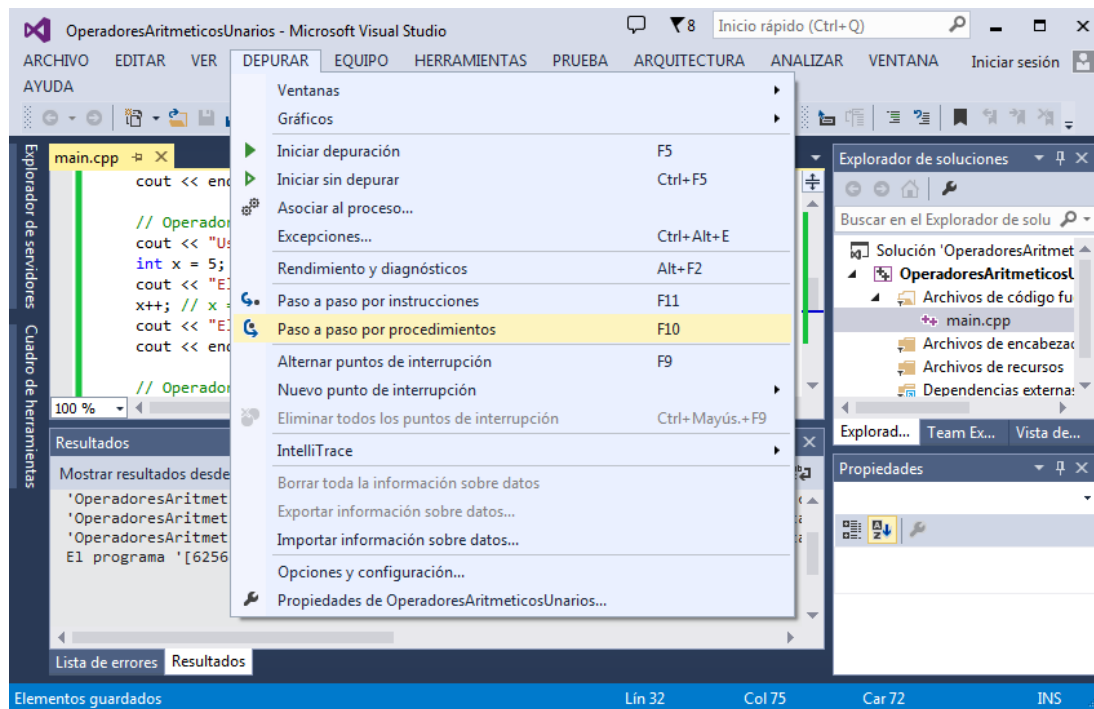


Figura 1.22. Herramienta de depuración.

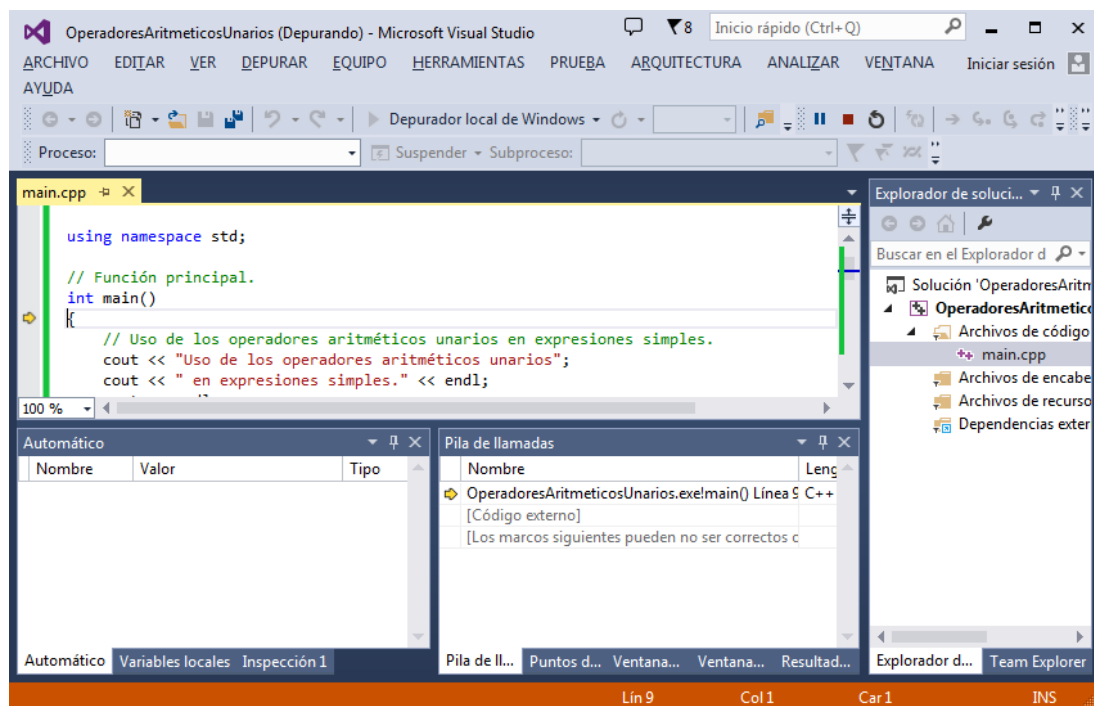


Figura 1.23. Depuración paso a paso por procedimientos.

Lenguaje C/C++. Problemas, Casos y Proyectos

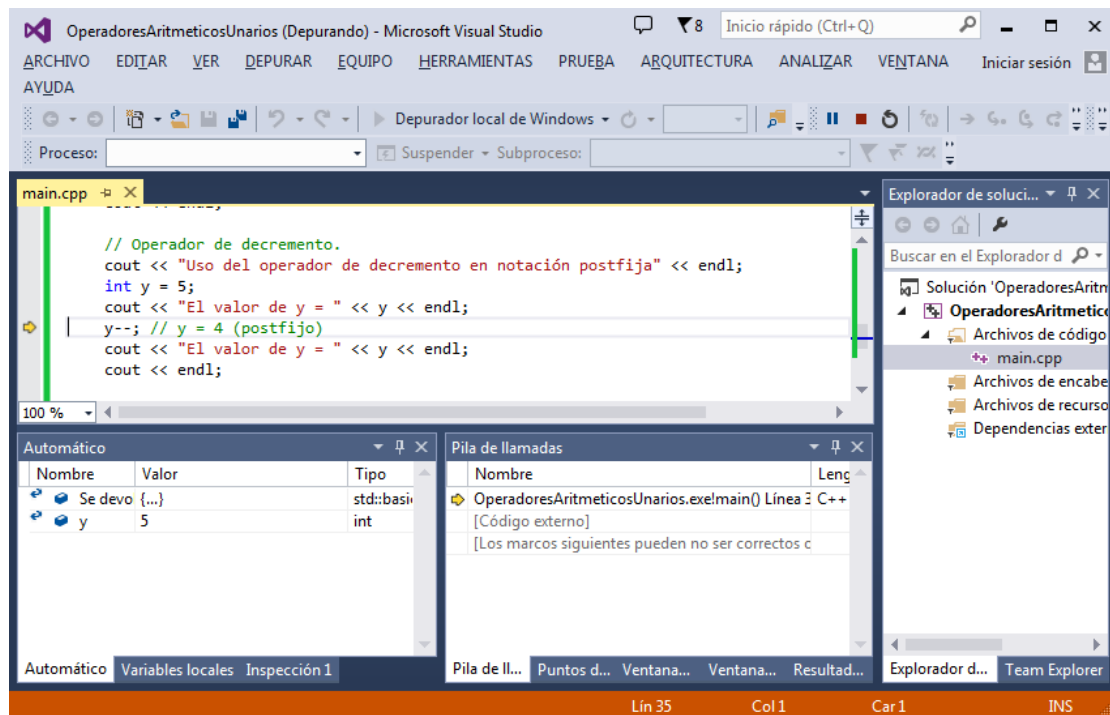


Figura 1.24. Prueba de escritorio con la computadora.

La opción de depuración de paso a paso por procedimientos, permite que el compilador ejecute paso a paso el programa desde el inicio hasta el final del mismo, presionando cada vez la tecla F10 sin entrar al detalle de las funciones, como se muestra en la Figura 1.23. Existe también la opción de paso a paso por instrucciones que se activa presionando la tecla F11, la cual hace lo mismo que la opción anterior, pero entrando a ver el detalle de cada una de las funciones, lo cual servirá cuando se estudie el capítulo correspondiente a funciones.

Una vez que se haya activado la opción de depuración de paso a paso por procedimientos, se puede hacer una prueba de escritorio utilizando la computadora, presionando F10 para pasar de una instrucción a otra, pero por procedimiento, donde en una ventana de depuración se puede ver los diferentes valores que van tomando las variables del programa, tal como se lo hace en la prueba de escritorio manual. La Figura 1.24 muestra los valores de las variables en el “Paso 6” de la prueba de escritorio con la computadora.

Problema 1.19: Escribir un programa que permita probar el uso de los operadores aritméticos unarios en expresiones matemáticas simples, utilizando las funciones ‘printf’ y ‘scanf’.

Programa 1.19: Código del programa.

```
/* *****  
WinConsolaPrograma_1_19  
***** */  
  
// Librerías.  
#include <stdio.h>  
#include <cstdlib>  
  
// Función principal.  
int main()  
{  
    // Uso de los operadores aritméticos unarios en expresiones simples.  
    printf("Uso de los operadores aritméticos unarios");  
    printf(" en expresiones simples.\n");  
    printf("\n");  
  
    // Operador de negación.
```

```
printf("Uso del operador de negación.\n");
int a = 5;
printf("El valor de a = %d\n", a);
int b = -a; // b = -5
printf("El valor de b = %d\n", b);
printf("\n");

// Operador de incremento.
printf("Uso del operador de incremento.\n");
int x = 5;
printf("El valor de x = %d\n", x);
++x; // x = 6 (prefijo)
printf("El valor de ++x = %d\n", x);
x++; // x = 7 (postfijo)
printf("El valor de x++ = %d\n", x);
printf("\n");

// Operador de decremento.
printf("Uso del operador de decremento.\n");
int y = 5;
printf("El valor de y = %d\n", y);
--y; // y = 4 (prefijo)
printf("El valor de --y = %d\n", y);
y--; // y = 3 (postfijo)
printf("El valor de y-- = %d\n", y);
printf("\n");

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.19: Salida del programa.

Tabla 1.19: Prueba de escritorio manual del programa.

Nº Paso	a	b	x	y
1	5			
2		-5		
3			5	
4			6	
5			7	
6				5
7				4
8				3

Problema 1.20: Escribir un programa que permita probar el uso de los operadores aritméticos unarios en expresiones matemáticas complejas, utilizando las funciones 'cout' y 'cin'.

Programa 1.20: Código del programa.

```
/******  
WinConsolaPrograma_1_20  
******/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // Declaración de variables.  
    int a, b, c, d;  
    int x, y, z, w;  
  
    // Paso 1.  
    a = 5; b = 2; c = 0; d = 0;  
    x = -3; y = -4; z = 1; w = -1;  
  
    cout << "Uso de los operadores aritméticos unarios";  
    cout << " en expresiones complejas." << endl;  
    cout << endl;  
    cout << "a = " << a << ", b = " << b << ", c = " << c << ", d = " << d << endl;  
    // a = 5; b = 2; c = 0; d = 0;  
    cout << endl;  
    cout << "x = " << x << ", y = " << y << ", z = " << z << ", w = " << w << endl;  
    // x = -3; y = -4; z = 1; w = -1;  
    cout << endl;  
  
    // Paso 2.  
    y = (++y) * (b--); // y = -3 * 2; y = -6;  
    cout << "y = " << y << endl;  
    // Paso 3.  
    x = (a++) * (--x); // x = 5 * (-4); x = -20;  
    cout << "x = " << x << endl;  
    // Paso 4.  
    z = (x++) + (++y); // z = (-20) + (-5); z = -25  
    cout << "z = " << z << endl;  
    cout << endl;  
    cout << "x = " << x << ", y = " << y << ", z = " << z << endl;  
    // x = -19; y = -5; z = -25  
    cout << endl;  
    // Paso 5.  
    c = (--a) - (b++); // z = 5 - 1; z = 4;  
    cout << "c = " << c << endl;  
    // Paso 6.  
    d = (z++) / (--c); // z = (-25) / 3; z = -8;  
    cout << "d = " << d << endl;  
    // Paso 7.  
    w = (a++) % (++b); // w = 5 % 3; w = 2;  
    cout << "w = " << w << endl;  
    // Paso 8.  
    z = (++x) + (y--) + (++z); // z = (-18) + (-5) + (-23); z = -46  
    cout << "z = " << z << endl;  
    cout << endl;  
}
```


Lenguaje C/C++. Problemas, Casos y Proyectos

```
// Paso 9.
cout << "a = " << a << ", b = " << b << ", c = " << c << ", d = " << d << endl;
// a = 6; b = 3; c = 3; d = -8;
cout << endl;
// Paso 10.
cout << "x = " << x << ", y = " << y << ", z = " << z << ", w = " << w << endl;
// x = -18; y = -6; z = -46; w = 2;
cout << endl;

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.20: Salida del programa.

Tabla 1.20: Prueba de escritorio manual del programa.

Nº Paso	Variables							
	a	b	c	d	x	y	z	w
1	5	2	0	0	-3	-4	1	-1
2		1			-	-3*2=-6		
3	6				5*(-4)=-20			
4					-19	-5	-20+(-5)=-25	
5	5	2	5-1=4					
6			3	-25/3=-8			-24	
7	6	3						5%3=2
8					-18	-6	(-18)+(-5)+(-23)=-46	
9	6	3	3	-8				
10					-18	-6	-46	2

Problema 1.21: Escribir un programa que permita probar el uso de los operadores aritméticos unarios en expresiones matemáticas complejas, utilizando las funciones 'printf' y 'scanf'.

Programa 1.21: Código del programa.

```
/******
WinConsolaPrograma_1_21
*****/

// Librerías.
```



```
#include <stdio.h>
#include <cstdlib>

// Función principal.
int main()
{
    int a, b, c, d;
    int x, y, z, w;

    // Paso 1.
    a = 5; b = 2; c = 0; d = 0;
    x = -3; y = -4; z = 1; w = -1;

    printf("Uso de los operadores aritméticos unarios");
    printf(" en expresiones complejas.\n");
    printf("\n");
    printf("a = %d, b = %d, c = %d, d = %d\n", a, b, c, d);
    // a = 5; b = 2; c = 0; d = 0;
    printf("\n");
    printf("x = %d, y = %d, z = %d, w = %d\n", x, y, z, w);
    // x = -3; y = -4; z = 1; w = -1;
    printf("\n");

    // Paso 2.
    y = (++y) * (b--); // y = -3 * 2; y = -6;
    printf("y = %d\n", y);
    // Paso 3.
    x = (a++) * (--x); // x = 5 * (-4); x = -20;
    printf("x = %d\n", x);
    // Paso 4.
    z = (x++) + (++y); // z = (-20) + (-5); z = -25
    printf("z = %d\n", z);
    printf("\n");
    printf("x = %d, y = %d, z = %d\n", x, y, z);
    // x = -19; y = -5; z = -25
    printf("\n");
    // Paso 5.
    c = (--a) - (b++); // z = 5 - 1; z = 4;
    printf("c = %d\n", c);
    // Paso 6.
    d = (z++) / (--c); // z = (-25) / 3; z = -8;
    printf("d = %d\n", d);
    // Paso 7.
    w = (a++) % (++b); // w = 5 % 3; w = 2;
    printf("w = %d\n", w);
    // Paso 8.
    z = (++x) + (y--) + (++z); // z = (-18) + (-5) + (-23); z = -46
    printf("z = %d\n", z);
    printf("\n");

    // Paso 9.
    printf("a = %d, b = %d, c = %d, d = %d\n", a, b, c, d);
    // a = 6; b = 3; c = 3; d = -8;
    printf("\n");
    // Paso 10.
    printf("x = %d, y = %d, z = %d, w = %d\n", x, y, z, w);
    // x = -18; y = -6; z = -46; w = 2;
    printf("\n");

    // Incorporar una pausa en el programa.
    system("pause");
    return 0;
}
```

```
}
```

Salida 1.21: Salida del programa.

```

C:\Users\cxvil\OneDrive\Documentos\Visual Studio 2019\Code Snippets\Visual C++\WinCons...
Uso de los operadores aritméticos unarios en expresiones complejas.

a = 5, b = 2, c = 0, d = 0

x = -3, y = -4, z = 1, w = -1

y = -6
x = -20
z = -25

x = -19, y = -5, z = -25

c = 4
d = -8
w = 2
z = -46

a = 6, b = 3, c = 3, d = -8

x = -18, y = -6, z = -46, w = 2

Press any key to continue . . .
    
```

Tabla 1.21: Prueba de escritorio manual del programa.

Nº Paso	Variables							
	a	b	c	d	x	y	z	w
1	5	2	0	0	-3	-4	1	-1
2		1			-	-3*2=-6		
3	6				5*(-4)=-20			
4					-19	-5	-20+(-5)=-25	
5	5	2	5-1=4					
6			3	-25/3=-8			-24	
7	6	3						5%3=2
8					-18	-6	(-18)+(-5)+(-23)=-46	
9	6	3	3	-8				
10					-18	-6	-46	2

1.5.6. Precedencia de Operadores Aritméticos

Considere la siguiente sentencia:

```
int x = 7 + 2 * 6 / 3 - 1;
```

En este punto surge una pregunta: ¿En qué orden el compilador ejecuta las varias operaciones aritméticas? En el lenguaje C/C++ cada operador tiene definida un nivel de precedencia, y los operadores son evaluados en un orden de mayor precedencia a menor precedencia. Las operaciones de multiplicación, división y módulo tienen el mismo nivel de precedencia. De manera similar la operación de suma y de resta, tienen el mismo nivel de precedencia. Por lo tanto, las operaciones de multiplicación, división y módulo tienen mayor precedencia que la suma y la resta, lo que implica que las operaciones de multiplicación, división y módulo siempre ocurrirán antes que la suma y la resta (Deitel, H., Deitel, P., 2003). Entonces, la expresión anterior será evaluada de la siguiente forma:

```

int x = 7 + 2 * 6 / 3 - 1;
      = 7 + 12 / 3 - 1;
      = 7 + 4 - 1;
    
```

```
= 11 - 1;  
= 10;
```

Nótese que los operadores con el mismo nivel de precedencia son evaluados de izquierda a derecha.

Algunas veces se requiere forzar a que una operación ocurra primero. Por ejemplo, en la expresión anterior, se pudo haber querido que la suma entre 7 y 2 y la resta entre 3 y 1 se realice primero. Así como está la expresión, no se puede cumplir con este caso, pero si encerramos entre paréntesis las expresiones que queremos que se evalúen primero, el problema está resuelto. Entonces se puede forzar la evaluación de operadores de menor precedencia como la suma y la resta sobre operadores de mayor precedencia como la multiplicación, la división y el módulo, utilizando paréntesis. Así entonces, tenemos reformulada la expresión matemática anterior, de la siguiente manera:

```
int x = (7 + 2) * 6 / (3 - 1);  
      = 9 * 6 / 2;  
      = 54 / 2;  
      = 27;
```

Los paréntesis también pueden estar anidados en una expresión matemática, donde usted puede especificar el grupo de expresiones que quiere que se evalúe primero, luego las que van en segundo lugar y después las que van en tercer lugar, etc. En una expresión con paréntesis anidados, las operaciones son evaluadas en cierto orden, desde el grupo de paréntesis más interno hasta el grupo de paréntesis más externo. El siguiente ejemplo muestra lo indicado:

```
int x = (((7 + 3) / 2) * (6 - 2)) - (18 / (7 + 2));  
      = ((10 / 2) * 4) - (18 / 9);  
      = (5 * 4) - 2;  
      = 20 - 2;  
      = 18;
```

Tips de Programación: Observe que los operadores aritméticos se utilizan para evaluar un valor numérico, que se conoce como expresión numérica. Generalizando, algo que evalúa a algo más se considera una expresión. En el siguiente capítulo veremos que también existen expresiones lógicas (expresiones con operadores lógicos), las cuales evalúan valores de veracidad o falsedad.

Los siguientes programas ilustran cómo trabaja la precedencia de operadores con expresiones numéricas.

Problema 1.22: Escribir un programa que permita probar el uso de la precedencia de operadores con las expresiones numéricas expuestas en la sección 1.5, utilizando las funciones 'cout' y 'cin'.

Programa 1.22: Código del programa.

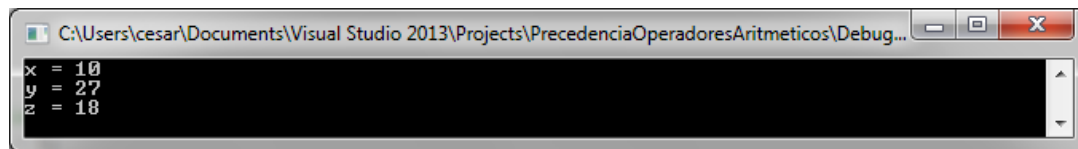
```
/******  
WinConsolaPrograma_1_22  
******/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // Declaración de variables y definición de expresiones.
```

```
int x = 7 + 2 * 6 / 3 - 1;
int y = (7 + 2) * 6 / (3 - 1);
int z = (((7 + 3) / 2) * (6 - 2)) - (18 / (7 + 2));

// Imprimir los valores de las expresiones numéricas.
cout << "x = " << x << endl;
cout << "y = " << y << endl;
cout << "z = " << z << endl;

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.22: Salida del programa.



Problema 1.23: Escribir un programa que permita probar el uso de la precedencia de operadores con las expresiones numéricas expuestas en la sección 1.5, utilizando las funciones 'printf' y 'scanf'.

Programa 1.23: Código del programa.

```
/* *****
WinConsolaPrograma_1_23
***** */

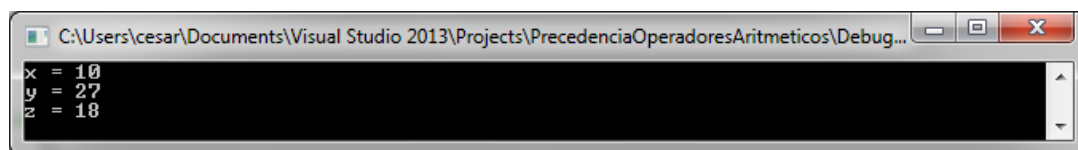
// Librerías.
#include <stdio.h>
#include <stdlib.h>

// Función principal.
int main()
{
    // Declaración de variables y definición de expresiones.
    int x = 7 + 2 * 6 / 3 - 1;
    int y = (7 + 2) * 6 / (3 - 1);
    int z = (((7 + 3) / 2) * (6 - 2)) - (18 / (7 + 2));

    // Imprimir los valores de las expresiones numéricas.
    printf("x = %d\n", x);
    printf("y = %d\n", y);
    printf("z = %d\n", z);

    // Incorporar una pausa en el programa.
    system("pause");
    return 0;
}
```

Salida 1.23: Salida del programa.



1.6. Operadores Relacionales

Los operadores relacionales se emplean para comparar o determinar alguna relación elemental entre dos variables o también entre dos expresiones y generar un valor de verdadero (1) o de falso (0) (Jones, B., Aitken, P., 2003). La Tabla 1.5 resume los operadores relacionales que maneja el lenguaje C/C++.

Tabla 1.5. Operadores Relacionales.

Operador	Significado	Descripción	Ejemplo
==	Igual a	El operador de igualdad que se usa en el lenguaje C/C++ es el signo de doble igual '==', puesto que el signo de simple igualdad '=' es el operador de asignación. Este operador retorna el valor de verdadero si los dos operandos o expresiones son iguales, caso contrario retorna el valor de falso.	a == b
!=	No igual a	El operador de desigualdad retorna el valor de verdadero si los dos operandos o expresiones son desiguales, caso contrario retorna el valor de falso.	a != b
>	Mayor que	El operador de mayor que retorna verdadero si el operando o la expresión de la izquierda es mayor que el operando o la expresión de la derecha, caso contrario retorna el valor de falso.	a > b
<	Menor que	El operador de menor que retorna verdadero si el operando o la expresión de la izquierda es menor que el operando o la expresión de la derecha, caso contrario retorna el valor de falso.	a < b
>=	Mayor o igual que	El operador de mayor o igual que retorna verdadero si el operando o la expresión de la izquierda es mayor o igual que el operando o la expresión de la derecha, caso contrario retorna el valor de falso.	a >= b
<=	Menor o igual que	El operador de menor o igual que retorna verdadero si el operando o la expresión de la izquierda es menor o igual que el operando o la expresión de la derecha, caso contrario retorna el valor de falso.	a <= b

Los siguientes programas ilustran cómo trabajan estos operadores relacionales.

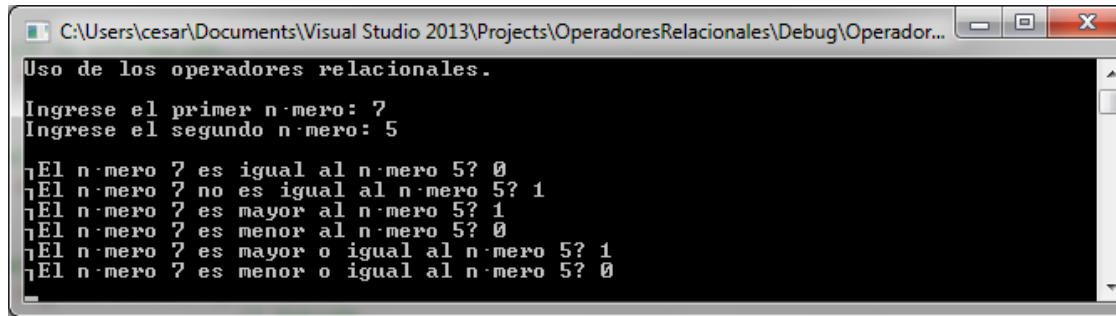
Problema 1.24: Escribir un programa que permita probar el uso de los operadores relacionales, con dos números enteros leídos desde el teclado, utilizando las funciones 'cout' y 'cin'.

Programa 1.24: Código del programa.

```
/* *****  
WinConsolaPrograma_1_24  
***** */  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
  
using namespace std;  
  
// Función principal.  
int main()
```

```
{  
    // declaración de variables.  
    int a = 0, b = 0;  
  
    bool EsIgual;  
    bool EsDesigual;  
    bool EsMayor;  
    bool EsMenor;  
    bool EsMayorOigual;  
    bool EsMenorOigual;  
  
    // Imprimir un mensaje de información.  
    cout << "Uso de los operadores relacionales." << endl;  
    cout << endl;  
  
    // Imprimir un mensaje y leer un dato de tipo entero.  
    cout << "Ingrese el primer número: "; // Salida.  
    cin >> a;                             // Entrada.  
  
    // Imprimir un mensaje y leer un dato de tipo entero.  
    cout << "Ingrese el segundo número: "; // Salida.  
    cin >> b;                             // Entrada.  
  
    // Imprimir un salto de línea (INTRO).  
    cout << endl;  
  
    // Almacenar en las variables booleanas las diferentes comparaciones.  
    EsIgual = a == b;  
    EsDesigual = a != b;  
    EsMayor = a > b;  
    EsMenor = a < b;  
    EsMayorOigual = a >= b;  
    EsMenorOigual = a <= b;  
  
    // Imprimir los valores de verdad o de falsedad de  
    // las diferentes comparaciones.  
    cout << "¿El número " << a << " es igual al número " << b << "? ";  
    cout << EsIgual << endl;  
    cout << "¿El número " << a << " no es igual al número " << b << "? ";  
    cout << EsDesigual << endl;  
    cout << "¿El número " << a << " es mayor al número " << b << "? ";  
    cout << EsMayor << endl;  
    cout << "¿El número " << a << " es menor al número " << b << "? ";  
    cout << EsMenor << endl;  
    cout << "¿El número " << a << " es mayor o igual al número " << b << "? ";  
    cout << EsMayorOigual << endl;  
    cout << "¿El número " << a << " es menor o igual al número " << b << "? ";  
    cout << EsMenorOigual << endl;  
  
    // Incorporar una pausa en el programa.  
    system("pause");  
    return 0;  
}
```

Salida 1.24: Salida del programa.



Problema 1.25: Escribir un programa que permita probar el uso de los operadores relacionales, con dos números enteros leídos desde el teclado, utilizando las funciones 'printf' y 'scanf'.

Programa 1.25: Código del programa.

```
/* **** */
WinConsolaPrograma_1_25
/* **** */

// Librerías.
#include <stdio.h>
#include <stdlib.h>

// Función principal.
int main()
{
    // declaración de variables.
    int a = 0, b = 0;

    int EsIgual;
    int EsDesigual;
    int EsMayor;
    int EsMenor;
    int EsMayorOigual;
    int EsMenorOigual;

    // Imprimir un mensaje de información.
    printf("Uso de los operadores relacionales.\n");
    printf("\n");

    // Imprimir un mensaje y leer un dato de tipo entero.
    printf("Ingrese el primer número: "); // Salida.
    scanf_s("%d", &a); // Entrada.

    // Imprimir un mensaje y leer un dato de tipo entero.
    printf("Ingrese el segundo número: "); // Salida.
    scanf_s("%d", &b); // Entrada.

    // Imprimir un salto de línea (INTRO).
    printf("\n");

    // Almacenar en las variables booleanas las diferentes comparaciones.
    EsIgual = a == b;
    EsDesigual = a != b;
    EsMayor = a > b;
    EsMenor = a < b;
    EsMayorOigual = a >= b;
    EsMenorOigual = a <= b;

    // Imprimir los valores de verdad o de falsedad de
```



```
// las diferentes comparaciones.
printf("¿El número %d es igual al número %d? ", a, b);
printf("%d\n", EsIgual);
printf("¿El número %d no es igual al número %d? ", a, b);
printf("%d\n", EsDesigual);
printf("¿El número %d es mayor al número %d? ", a, b);
printf("%d\n", EsMayor);
printf("¿El número %d es menor al número %d? ", a, b);
printf("%d\n", EsMenor);
printf("¿El número %d es mayor o igual al número %d? ", a, b);
printf("%d\n", EsMayorOigual);
printf("¿El número %d es menor o igual al número %d? ", a, b);
printf("%d\n", EsMenorOigual);

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.25: Salida del programa.

1.7. Operadores Lógicos

Los operadores lógicos o puertas lógicas se utilizan con operandos o expresiones para devolver un valor de verdadero (1) o un valor de falso (0) (Jones, B., Aitken, P., 2003). A estos operadores también se los denomina como operadores booleanos, en honor de George Boole, creador del álgebra de Boole. La Tabla 1.6, 1.7 y 1.8 resumen las tablas de verdad de los tres operadores lógicos o puertas lógicas que maneja el lenguaje C/C++.

Tabla 1.6. Tabla de verdad del operador lógico AND (&&) o conjunción lógica ($A \wedge B$).

Operandos		Operador Lógico
A	B	A && B
Verdadero (1)	Verdadero (1)	Verdadero (1)
Verdadero (1)	Falso (0)	Falso (0)
Falso (0)	Verdadero (1)	Falso (0)
Falso (0)	Falso (0)	Falso (0)

Tabla 1.7. Tabla de verdad del operador lógico OR (||) o disyunción lógica ($A \vee B$).

Operandos		Operador Lógico
A	B	A B
Verdadero (1)	Verdadero (1)	Verdadero (1)
Verdadero (1)	Falso (0)	Verdadero (1)
Falso (0)	Verdadero (1)	Verdadero (1)
Falso (0)	Falso (0)	Falso (0)

Tabla 1.8. Tabla de verdad del operador lógico NOT (!).

Operando	Negación
A	!A
Verdadero (1)	Falso (0)
Falso (0)	Verdadero (1)

Adicionalmente a estas puertas lógicas, se puede implementar la puerta lógica XOR u OR exclusiva mediante la siguiente expresión algebraica: $(A \parallel B) \&\& (!A \parallel !B)$, cuya tabla de verdad resultante se muestra en la Tabla 1.9.

Tabla 1.9. Tabla de verdad de la puerta lógica XOR o disyunción exclusiva ($A \leftrightarrow B$).

Operandos		Operador Lógico
A	B	A XOR B
Verdadero (1)	Verdadero (1)	Falso (0)
Verdadero (1)	Falso (0)	Verdadero (1)
Falso (0)	Verdadero (1)	Verdadero (1)
Falso (0)	Falso (0)	Falso (0)

Los siguientes programas ilustran cómo trabajan estos operadores lógicos.

Problema 1.26: Escribir un programa que permita representar la expresión algebraica: $(A \parallel B) \&\& (!A \parallel !B)$ y probar una de las salidas de la puerta lógica XOR, para lo cual se debe leer por teclado el valor de verdad de A y de B, utilizando las funciones 'cout' y 'cin'.

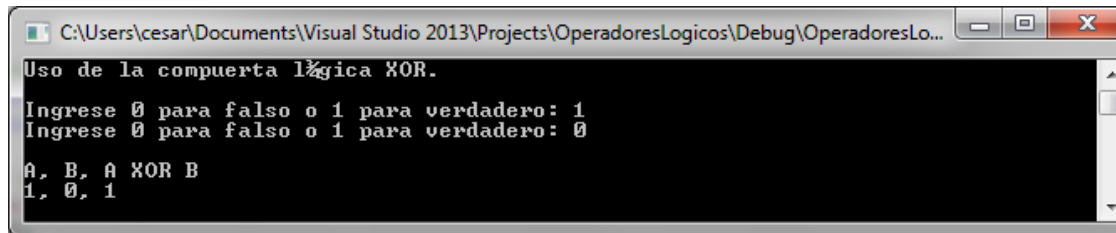
Programa 1.26: Código del programa.

```
/******  
WinConsolaPrograma_1_26  
*****/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // declaración de variables.  
    bool A = false;  
    bool B = false;  
  
    // Imprimir un mensaje de información.  
    cout << "Uso de la compuerta lógica XOR." << endl;  
    cout << endl;  
  
    // Imprimir un mensaje y leer un dato de tipo bool.  
    cout << "Ingrese 0 para falso o 1 para verdadero: "; // Salida.  
    cin >> A; // Entrada.  
  
    // Imprimir un mensaje y leer un dato de tipo bool.  
    cout << "Ingrese 0 para falso o 1 para verdadero: "; // Salida.  
    cin >> B; // Entrada.  
  
    // Imprimir un salto de línea (INTRO).  
    cout << endl;  
  
    // Evaluar la expresión algebraica equivalente al XOR  
    bool isXOR = (A || B) && (!A || !B);
```

```
// Imprimir el resultado obtenido según la tabla de verdad del XOR.
cout << "A, B, A XOR B" << endl;
cout << A << ", " << B << ", " << isXOR << endl;

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.26: Salida del programa.



Problema 1.27: Escribir un programa que permita representar la expresión algebraica: $(A \parallel B) \&\& (!A \parallel !B)$ y probar una de las salidas de la puerta lógica XOR, para lo cual se debe leer por teclado el valor de verdad de A y de B, utilizando las funciones 'printf' y 'scanf'.

Programa 1.27: Código del programa.

```
/* *****
WinConsolaPrograma_1_27
***** */

// Librerías.
#include <stdio.h>
#include <cstdlib>

// Función principal.
int main()
{
    // declaración de variables.
    int A = 0;
    int B = 0;

    // Imprimir un mensaje de información.
    printf("Uso de la compuerta lógica XOR.\n");
    printf("\n");

    // Imprimir un mensaje y leer un dato de tipo bool.
    printf("Ingrese 0 para falso o 1 para verdadero: "); // Salida.
    scanf_s("%d", &A); // Entrada.

    // Imprimir un mensaje y leer un dato de tipo bool.
    printf("Ingrese 0 para falso o 1 para verdadero: "); // Salida.
    scanf_s("%d", &B); // Entrada.

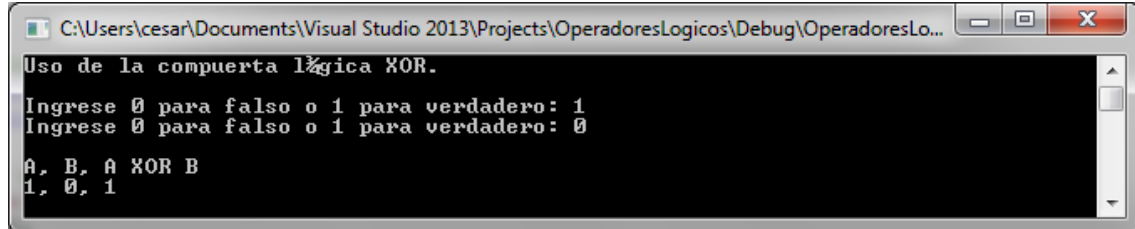
    // Imprimir un salto de línea (INTRO).
    printf("\n");

    // Evaluar la expresión algebraica equivalente al XOR
    int isXOR = (A || B) && (!A || !B);

    // Imprimir el resultado obtenido según la tabla de verdad del XOR.
    printf("A, B, A XOR B\n");
    printf("%d, %d, %d\n", A, B, isXOR);
}
```

```
// Incorporar una pausa en el programa.  
system("pause");  
return 0;  
}
```

Salida 1.27: Salida del programa.



1.8. Fórmulas Matemáticas

Adicionalmente al uso de expresiones aritméticas, el C/C++ permite estructurar fórmulas matemáticas para diferentes propósitos como: calcular el área de un círculo, calcular el volumen de un cilindro, calcular el tiempo de vuelo de un proyectil que sigue una trayectoria parabólica, etc. Todos estos problemas y más, se resuelven utilizando fórmulas matemáticas (Granizo, E., 2016). El C/C++ para esto tiene una librería de funciones matemáticas que pertenecen al archivo de cabecera 'cmath.h'. La Figura 1.25 muestra 5 de las ecuaciones consideradas que cambiaron el mundo (Guillen, M., 2000):

- | | |
|---|-----------------------------|
| 1) LEY DE GRAVITACIÓN UNIVERSAL: | (ISAAC NEWTON) |
| $F = G \frac{m_1 \cdot m_2}{d^2}$ | |
| 2) LEY DE LA PRESIÓN HIDRODINÁMICA: | (DANIEL BERNOULLI) |
| $P_1 + \frac{1}{2} \rho v_1^2 + \rho g h_1 = P_2 + \frac{1}{2} \rho v_2^2 + \rho g h_2$ | |
| 3) LEY DE LA INDUCCIÓN ELECTROMAGNÉTICA: | (MICHAEL FARADAY) |
| $\varepsilon = \frac{d\Phi}{dt}$ | |
| 4) SEGUNDA LEY DE LA TERMODINÁMICA: | (RUDOLF CLAUSIUS) |
| $\Delta S = \frac{\Delta Q}{T}$ | |
| 5) TEORÍA DE LA RELATIVIDAD ESPECIAL: | (ALBERT EINSTEIN) |
| $E = m \cdot c^2$ | |

Figura 1.25. Cinco ecuaciones que cambiaron el mundo (Guillen, M., 2000).

1.8.1. Funciones de la Librería cmath

Como se mencionó en la sección 1.2.1, la librería "cmath" es un alias de "math.h" o la Librería de Encabezado de Funciones Matemáticas (Math Library Header), que es una librería que tiene funciones para realizar operaciones matemáticas básicas, tales como: funciones trigonométricas, funciones exponenciales, funciones logarítmicas, funciones para calcular la raíz cuadrada, funciones para calcular el valor absoluto de un número, etc., (Luna, F., 2004). La Tabla 1.10 resume algunas de las funciones matemáticas más comúnmente utilizadas:

Tabla 1.10. Algunas de las Funciones Matemáticas de la Librería 'cmath'.

Declaración de la Función	Descripción
double sqrt(double x);	Retorna $\sqrt{x}$.
double pow(double x, double y);	Retorna x^y .
double fabs(double x);	Retorna $ x $.

<code>double exp(double x);</code>	Retorna e^x .
<code>double log(double x);</code>	Retorna $\ln(x)$.
<code>double cos(double x);</code>	Retorna $\cos(x)$.
<code>double sin(double x);</code>	Retorna $\sin(x)$.
<code>double tan(double x);</code>	Retorna $\tan(x)$.
<code>double acos(double x);</code>	Retorna $\arccos(x)$.
<code>double asin(double x);</code>	Retorna $\arcsen(x)$.
<code>double atan(double x);</code>	Retorna $\arctan(x)$.
<code>double atan2(double y, double x);</code>	Retorna $\arctan(y/x)$.

Las funciones trigonométricas sólo trabajan con radianes y no con grados. Un número 'x' puede ser convertido des grados a radianes multiplicando ese valor por $\pi/180^\circ$. Así mismo, un número 'x' puede ser convertido de radianes a grados multiplicando ese valor por $180^\circ/\pi$. Por ejemplo: $360 = 360^\circ \times \pi/180^\circ = 2\pi$.

Las funciones mencionadas anteriormente trabajan con flotantes de doble precisión (*double*). En el mundo real, el tipo de dato más utilizado en la mayor parte de las aplicaciones científicas y matemáticas es el *float* (*flotante*), por lo que si se quiere trabajar con estas funciones es necesario realizar conversiones de tipos de datos implícitas.

Los siguientes programas ilustran cómo trabajan las diferentes funciones matemáticas.

Problema 1.28: Escribir un programa que permita probar el uso de al menos seis de las doce funciones matemáticas de la Tabla 1.10, utilizando las funciones 'cout' y 'cin'.

Programa 1.28: Código del programa.

```

/*****
WinConsolaPrograma_1_28
*****/

// Librerías.
#include <iostream>
#include <cstdlib>
#include <cmath>

using namespace std;

// Función principal.
int main()
{
    // Declaración de variables.
    float x = 0.0f;

    // Imprimir un mensaje de información.
    cout << "Manejo de funciones matemáticas." << endl;
    cout << endl;

    // Imprimir un mensaje y leer un dato de tipo flotante.
    cout << "Ingrese el valor de x: "; // Salida.
    cin >> x;                          // Entrada.

    // Imprimir un salto de línea (INTRO).
    cout << endl;

    // Realizar cálculos con las funciones matemáticas de C/C++.
    cout << "cos(" << x << ") = " << cos(x) << endl;
    cout << "sen(" << x << ") = " << sin(x) << endl;
    cout << "raiz_cuadrada(" << x << ") = " << sqrt(x) << endl;
    cout << "log_natural(" << x << ") = " << log(x) << endl;
}

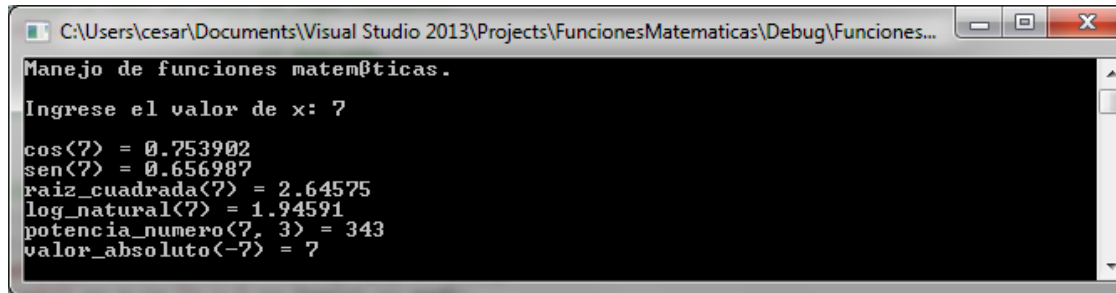
```

Lenguaje C/C++. Problemas, Casos y Proyectos

```
cout << "potencia_numero(" << x << ", 3) = " << pow(x, 3) << endl;
cout << "valor_absoluto(" << -x << ") = " << fabs(-x) << endl;

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.28: Salida del programa.



Problema 1.29: Escribir un programa que permita probar el uso de al menos seis de las doce funciones matemáticas de la Tabla 1.10, utilizando las funciones 'printf' y 'scanf'.

Programa 1.29: Código del programa.

```
/* *****
WinConsolaPrograma_1_29
***** */

// Librerías.
#include <stdio.h>
#include <stdlib.h>
#include <cmath>

// Función principal.
int main()
{
    // Declaración de variables.
    float x = 0.0f;

    // Imprimir un mensaje de información.
    printf("Manejo de funciones matemáticas.\n");
    printf("\n");

    // Imprimir un mensaje y leer un dato de tipo flotante.
    printf("Ingrese el valor de x : "); // Salida.
    scanf_s("%f", &x); // Entrada.

    // Imprimir un salto de línea (INTRO).
    printf("\n");

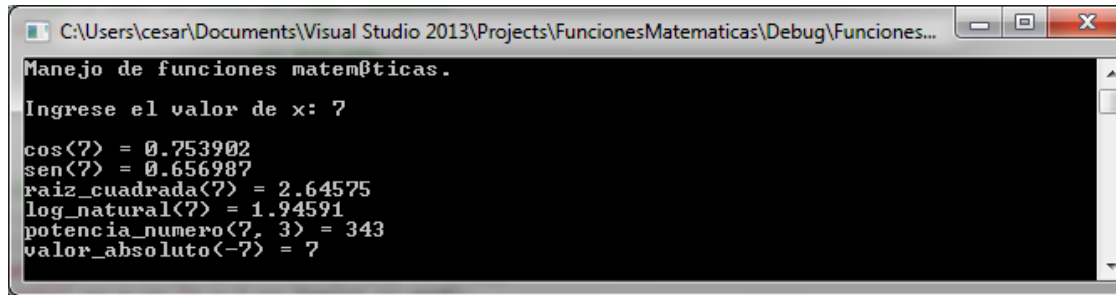
    // Realizar cálculos con las funciones matemáticas de C/C++.
    printf("cos(%f) = %f\n", x, cos(x));
    printf("sen(%f) = %f\n", x, sin(x));
    printf("raiz_cuadrada(%f) = %f\n", x, sqrt(x));
    printf("log_natural(%f) = %f\n", x, log(x));
    printf("potencia_numero(%f, 3) = %f\n", x, pow(x, 3));
    printf("valor_absoluto(-%f) = %f\n", x, fabs(-x));

    // Incorporar una pausa en el programa.
    system("pause");
    return 0;
}
```



```
}
```

Salida 1.29: Salida del programa.



Problema 1.30: Escribir un programa que calcule el área y el perímetro de un cuadrado, utilizando las funciones 'cout', 'cin' y las siguientes fórmulas matemáticas:

$$\begin{array}{ll} p = 4l & (1) \text{ Perímetro del cuadrado} \\ \text{área} = l^2 & (2) \text{ Área del cuadrado} \end{array}$$

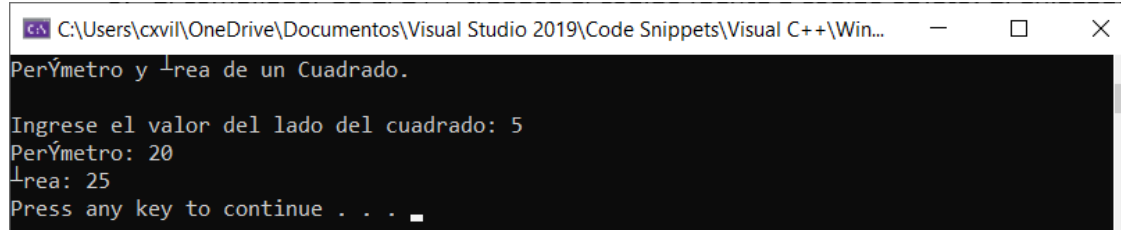
Siendo 'l' el valor del lado del cuadrado. Además, trabajar con valores flotantes y con las funciones matemáticas de la librería 'cmath'.

Programa 1.30: Código del programa.

```
/******  
WinConsolaPrograma_1_30  
*****/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
#include <cmath>  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // Declaración de variables.  
    float lado; // Entrada: Lado del cuadrado.  
    float p;    // Salida: Perímetro del cuadrado.  
    float area; // Salida: Área del cuadrado.  
  
    // Imprimir un mensaje de información y un salto de línea.  
    cout << "Perímetro y Área de un Cuadrado." << endl;  
    // Imprimir un salto de línea (INTRO).  
    cout << endl;  
  
    // Imprimir un mensaje y leer el lado del cuadrado.  
    cout << "Ingrese el valor del lado del cuadrado: ";  
    cin >> lado;  
  
    // Calcular el perímetro del cuadrado.  
    p = 4 * lado;  
    // Calcular el área del cuadrado.  
    area = pow(lado, 2);  
  
    // Imprimir el perímetro y el área del cuadrado.  
    cout << "Perímetro: " << p << endl;  
    cout << "Área: " << area << endl;  
}
```

```
// Incorporar una pausa en el programa.  
system("pause");  
return 0;  
}
```

Salida 1.30: Salida del programa.



Problema 1.31: Escribir un programa que calcule el área y el perímetro de un cuadrado, utilizando las funciones 'printf', 'scanf' y las siguientes fórmulas matemáticas:

$$\begin{array}{ll} p = 4l & (1) \quad \text{Perímetro del cuadrado} \\ \text{área} = l^2 & (2) \quad \text{Área del cuadrado} \end{array}$$

Siendo 'l' el valor del lado del cuadrado. Además, trabajar con valores flotantes y con las funciones matemáticas de la librería 'cmath'.

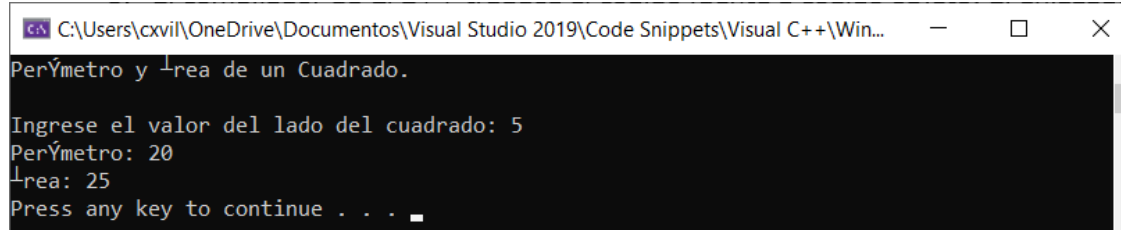
Programa 1.31: Código del programa.

```
/* *****  
WinConsolaPrograma_1_31  
***** */  
  
// Librerías.  
#include <stdio.h>  
#include <stdlib.h>  
#include <cmath>  
  
// Función principal.  
int main()  
{  
    // Declaración de variables.  
    float lado; // Entrada: Lado del cuadrado.  
    float p;    // Salida: Perímetro del cuadrado.  
    float area; // Salida: Área del cuadrado.  
  
    // Imprimir un mensaje de información y un salto de línea.  
    printf("Perímetro y Área de un Cuadrado.\n");  
    // Imprimir un salto de línea (INTRO).  
    printf("\n");  
  
    // Imprimir un mensaje y leer el lado del cuadrado.  
    printf("Ingrese el valor del lado del cuadrado: ");  
    scanf_s("%f", &lado);  
  
    // Calcular el perímetro del cuadrado.  
    p = 4 * lado;  
    // Calcular el área del cuadrado.  
    area = pow(lado, 2);  
  
    // Imprimir el perímetro y el área del cuadrado.  
    printf("Perímetro: %f\n", p);  
    printf("Área: %f\n", area);  
}
```



```
// Incorporar una pausa en el programa.  
system("pause");  
return 0;  
}
```

Salida 1.31: Salida del programa.



Problema 1.32: Escribir un programa que calcule el área y el perímetro de un rectángulo, utilizando las funciones 'cout', 'cin' y las siguientes fórmulas matemáticas:

$$\begin{array}{ll} p = 2a + 2b & (1) \text{ Perímetro del rectángulo} \\ \text{área} = a \cdot b & (2) \text{ Área del rectángulo} \end{array}$$

Siendo 'a' y 'b' los valores de los lados del rectángulo. (Trabajar con valores flotantes)

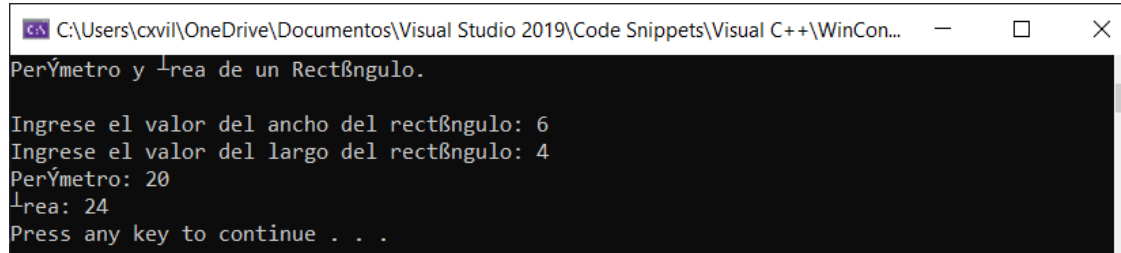
Programa 1.32: Código del programa.

```
/*  
WinConsolaPrograma_1_32  
*/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
#include <cmath>  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // Declaración de variables.  
    float a;    // Entrada: Ancho del rectángulo.  
    float b;    // Entrada: Largo del rectángulo.  
    float p;    // Salida: Perímetro del rectángulo.  
    float area; // Salida: Área del rectángulo.  
  
    // Imprimir un mensaje de información y un salto de línea.  
    cout << "Perímetro y Área de un Rectángulo." << endl;  
    // Imprimir un salto de línea (INTRO).  
    cout << endl;  
  
    // Imprimir un mensaje y leer el ancho del rectángulo.  
    cout << "Ingrese el valor del ancho del rectángulo: ";  
    cin >> a;  
    // Imprimir un mensaje y leer el largo del rectángulo.  
    cout << "Ingrese el valor del largo del rectángulo: ";  
    cin >> b;  
  
    // Calcular el perímetro del rectángulo.  
    p = 2 * a + 2 * b;  
    // Calcular el área del rectángulo.  
    area = a * b;  
}
```

```
// Imprimir el perímetro y el área del rectángulo.
cout << "Perímetro: " << p << endl;
cout << "Área: " << area << endl;

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.32: Salida del programa.



Problema 1.33: Escribir un programa que calcule el área y el perímetro de un rectángulo, utilizando las funciones 'printf', 'scanf' y las siguientes fórmulas matemáticas:

$$\begin{array}{ll} p = 2a + 2b & (1) \quad \text{Perímetro del rectángulo} \\ \text{área} = a \cdot b & (2) \quad \text{Área del rectángulo} \end{array}$$

Siendo 'a' y 'b' los valores de los lados del rectángulo. (Trabajar con valores flotantes)

Programa 1.33: Código del programa.

```
/* *****
WinConsolaPrograma_1_33
***** */

// Librerías.
#include <stdio.h>
#include <stdlib.h>
#include <cmath>

// Función principal.
int main()
{
    // Declaración de variables.
    float a;    // Entrada: Ancho del rectángulo.
    float b;    // Entrada: Largo del rectángulo.
    float p;    // Salida: Perímetro del rectángulo.
    float area; // Salida: Área del rectángulo.

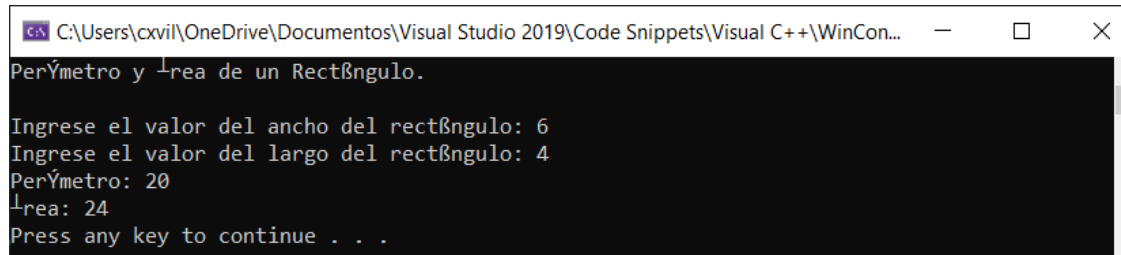
    // Imprimir un mensaje de información y un salto de línea.
    printf("Perímetro y Área de un Rectángulo.\n");
    // Imprimir un salto de línea (INTRO).
    printf("\n");

    // Imprimir un mensaje y leer el ancho del rectángulo.
    printf("Ingrese el valor del ancho del rectángulo: ");
    scanf_s("%f", &a);
    // Imprimir un mensaje y leer el largo del rectángulo.
    printf("Ingrese el valor del largo del rectángulo: ");
    scanf_s("%f", &b);

    // Calcular el perímetro del rectángulo.
```

```
p = 2 * a + 2 * b;  
// Calcular el área del rectángulo.  
area = a * b;  
  
// Imprimir el perímetro y el área del rectángulo.  
printf("Perímetro: %f\n", p);  
printf("Área: %f\n", area);  
  
// Incorporar una pausa en el programa.  
system("pause");  
return 0;  
}
```

Salida 1.33: Salida del programa.



Problema 1.34: Escribir un programa para calcular e imprimir el perímetro y el área de un círculo, utilizando las funciones 'cout', 'cin' y las siguientes fórmulas matemáticas:

$$\begin{array}{ll} p = 2\pi r & (1) \text{ Perímetro del círculo} \\ \text{área} = \pi r^2 & (2) \text{ Área del círculo} \end{array}$$

Siendo 'r' el valor del radio del círculo. (Trabajar con valores flotantes).

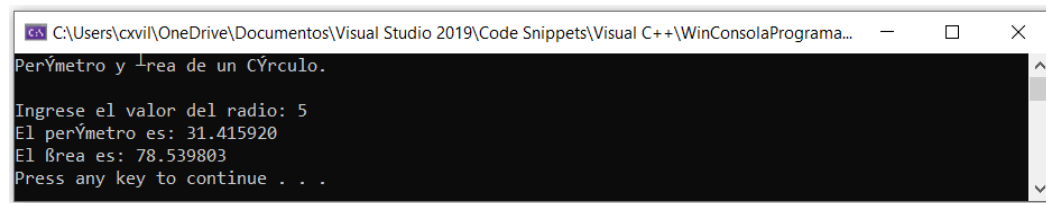
Programa 1.34: Código del programa.

```
/******  
WinConsolaPrograma_1_34  
******/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
#include <cmath>  
  
// Macros o sentencias de preprocesador.  
#define PI 3.141592  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // Declaración de variables.  
    float radio; // Entrada: radio de un círculo.  
    float perimetro; // Salida: perímetro de un círculo.  
    float area; // Salida: área de un círculo.  
  
    // Imprimir un mensaje de información y dos saltos de línea.  
    cout << "Perímetro y Área de un Círculo." << endl << endl;  
  
    // Leer el radio del círculo utilizando la variable 'radio'.  
    cout << "Ingrese el valor del radio: ";  
    cin >> radio;
```

```
// Calcular el perímetro.
perimetro = 2 * PI * radio;
// Calcular el área.
area = PI * pow(radio, 2);
// Imprimir el valor de la variable 'perimetro' y de la variable 'area'.
cout << "El perímetro es: " << perimetro << endl;
cout << "El área es: " << area << endl;

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.34: Salida del programa.



Problema 1.35: Escribir un programa para calcular e imprimir el perímetro y el área de un círculo, utilizando las funciones 'printf', 'scanf' y las siguientes fórmulas matemáticas:

$$\begin{array}{ll} p = 2\pi r & (1) \quad \text{Perímetro del círculo} \\ \text{área} = \pi r^2 & (2) \quad \text{Área del círculo} \end{array}$$

Siendo 'r' el valor del radio del círculo. (Trabajar con valores flotantes).

Programa 1.35: Código del programa.

```
/* *****
WinConsolaPrograma_1_35
***** */

// Librerías.
#include <stdio.h>
#include <stdlib.h>
#include <cmath>

// Macros o sentencias de preprocesador.
#define PI 3.141592

// Función principal.
int main()
{
    // Declaración de variables.
    float radio; // Entrada: radio de un círculo.
    float perimetro; // Salida: perímetro de un círculo.
    float area; // Salida: área de un círculo.

    // Imprimir un mensaje de información y dos saltos de línea.
    printf("Perímetro y Área de un Círculo.\n\n");

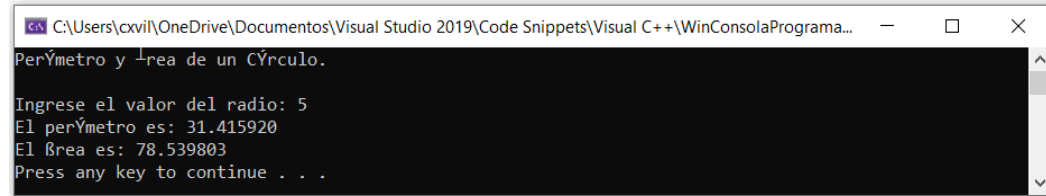
    // Leer el radio del círculo utilizando la variable 'radio'.
    printf("Ingrese el valor del radio: ");
    scanf_s("%f", &radio);
    // Calcular el perímetro.
    perimetro = 2 * PI * radio;
    // Calcular el área.
    area = PI * pow(radio, 2);
}
```

Lenguaje C/C++. Problemas, Casos y Proyectos

```
// Imprimir el valor de la variable 'perimetro' y de la variable 'area'.
printf("El perímetro es: %f\n", perimetro);
printf("El área es: %f\n", area);

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.35: Salida del programa.



Problema 1.36: Escribir un programa que calcule la hipotenusa y los dos ángulos internos de un triángulo rectángulo que suman 90°, utilizando las funciones 'cout', 'cin' y las siguientes fórmulas matemáticas:

$$\begin{aligned} c &= \sqrt{a^2 + b^2} & (1) & \text{Hipotenusa de un triángulo rectángulo} \\ \hat{A} &= \arctan\left(\frac{a}{b}\right) & (2) & \text{Primer ángulo del triángulo rectángulo} \\ \hat{B} &= \arctan\left(\frac{b}{a}\right) & (3) & \text{Segundo ángulo del triángulo rectángulo} \end{aligned}$$

Siendo 'a' y 'b' los valores de los catetos del triángulo rectángulo. (Trabajar con valores flotantes).

Programa 1.36: Código del programa.

```
/******
WinConsolaPrograma_1_36
******/

// Librerías.
#include <iostream>
#include <cstdlib>
#include <cmath>

// Macros o sentencias de preprocesador.
#define PI 3.141592

using namespace std;

// Función principal.
int main()
{
    // Declaración de variables.
    float a;           // Entrada: cateto 1 del triángulo rectángulo.
    float b;           // Entrada: cateto 2 del triángulo rectángulo.
    float hipotenusa; // Salida: hipotenusa del triángulo rectángulo.
    float anguloA;     // Salida: ángulo A del triángulo rectángulo.
    float anguloB;     // Salida: ángulo B del triángulo rectángulo.

    // Imprimir un mensaje de información y dos saltos de línea.
    cout << "Ángulos e Hipotenusa de un Triángulo Rectángulo." << endl << endl;

    // Leer el valor del cateto 1, utilizando la variable 'a'.
    cout << "Ingrese el valor del cateto 'a': ";
```

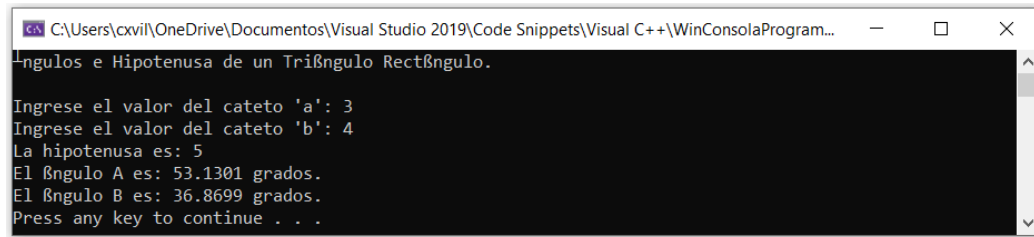
```
cin >> a;
// Leer el valor del cateto 2, utilizando la variable 'b'.
cout << "Ingrese el valor del cateto 'b': ";
cin >> b;

// Calcular la hipotenusa.
hipotenusa = sqrt(pow(a, 2) + pow(b, 2));
// Calcular el ángulo A del triángulo rectángulo.
anguloA = atan2(b, a);
// Convertir el valor del ángulo A de radianes a grados.
anguloA = anguloA * 180 / PI;
// Calcular el ángulo A del triángulo rectángulo.
anguloB = atan2(a, b);
// Convertir el valor del ángulo B de radianes a grados.
anguloB = anguloB * 180 / PI;

// Imprimir el valor de la variable 'hipotenusa'.
cout << "La hipotenusa es: " << hipotenusa << endl;
// Imprimir el valor de la variable 'anguloA'.
cout << "El ángulo A es: " << anguloA << " grados." << endl;
// Imprimir el valor de la variable 'anguloB'.
cout << "El ángulo B es: " << anguloB << " grados." << endl;

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.36: Salida del programa.



Problema 1.37: Escribir un programa que calcule la hipotenusa y los dos ángulos internos de un triángulo rectángulo que suman 90° , utilizando las funciones 'printf', 'scanf' y las siguientes fórmulas matemáticas:

$$\begin{aligned} c &= \sqrt{a^2 + b^2} & (1) & \text{ Hipotenusa de un triángulo rectángulo} \\ \hat{A} &= \arctan\left(\frac{a}{b}\right) & (2) & \text{ Primer ángulo del triángulo rectángulo} \\ \hat{B} &= \arctan\left(\frac{b}{a}\right) & (3) & \text{ Segundo ángulo del triángulo rectángulo} \end{aligned}$$

Siendo 'a' y 'b' los valores de los catetos del triángulo rectángulo. (Trabajar con valores flotantes).

Programa 1.37: Código del programa.

```
/******
WinConsolaPrograma_1_37
*****/

// Librerías.
#include <stdio.h>
#include <stdlib.h>
#include <math>
```

```
// Macros o sentencias de preprocesador.
#define PI 3.141592

// Función principal.
int main()
{
    // Declaración de variables.
    float a;           // Entrada: cateto 1 del triángulo rectángulo.
    float b;           // Entrada: cateto 2 del triángulo rectángulo.
    float hipotenusa;  // Salida: hipotenusa del triángulo rectángulo.
    float anguloA;     // Salida: ángulo A del triángulo rectángulo.
    float anguloB;     // Salida: ángulo B del triángulo rectángulo.

    // Imprimir un mensaje de información y dos saltos de línea.
    printf("Ángulos e Hipotenusa de un Triángulo Rectángulo.\n\n");

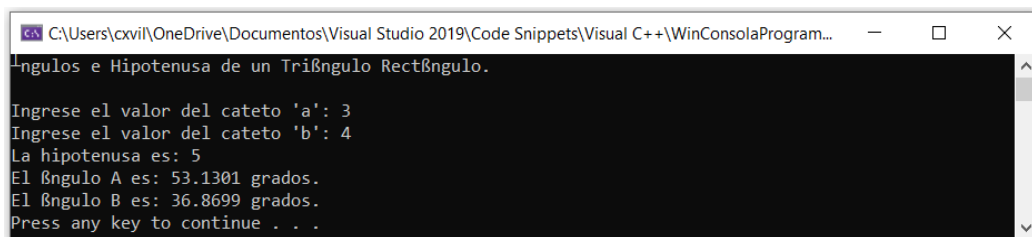
    // Leer el valor del cateto 1, utilizando la variable 'a'.
    printf("Ingrese el valor del cateto 'a': ");
    scanf_s("%f", &a);
    // Leer el valor del cateto 2, utilizando la variable 'b'.
    printf("Ingrese el valor del cateto 'b': ");
    scanf_s("%f", &b);

    // Calcular la hipotenusa.
    hipotenusa = sqrt(pow(a, 2) + pow(b, 2));
    // Calcular el ángulo A del triángulo rectángulo.
    anguloA = atan2(b, a);
    // Convertir el valor del ángulo A de radianes a grados.
    anguloA = anguloA * 180 / PI;
    // Calcular el ángulo B del triángulo rectángulo.
    anguloB = atan2(a, b);
    // Convertir el valor del ángulo B de radianes a grados.
    anguloB = anguloB * 180 / PI;

    // Imprimir el valor de la variable 'hipotenusa'.
    printf("La hipotenusa es: %f\n", hipotenusa);
    // Imprimir el valor de la variable 'anguloA'.
    printf("El ángulo A es: %f grados\n", anguloA);
    // Imprimir el valor de la variable 'anguloB'.
    printf("El ángulo B es: %f grados\n", anguloB);

    // Incorporar una pausa en el programa.
    system("pause");
    return 0;
}
```

Salida 1.37: Salida del programa.



Problema 1.38: Escribir un programa para calcular e imprimir la altura ($y = ?$) y la distancia entre el punto A y B ($h = ?$), donde se encuentra un blanco para práctica de tiro (B) en un plano inclinado cuya forma corresponde a un triángulo rectángulo como se muestra en la Figura 1.38.1. En la parte inferior correspondiente al punto A se encuentra un aprendiz de tiro al blanco cuya distancia en la base del triángulo es conocida. Se sabe además que el aprendiz dispara un proyectil con un ángulo θ ($theta$) de lanzamiento conocido con respecto a la horizontal y con una

Lenguaje C/C++. Problemas, Casos y Proyectos

velocidad V_0 conocida, de acuerdo a las siguientes fórmulas matemáticas. (Problema adaptado de Serway y Jewett (2005)).

$$y = x \cdot \tan(\theta) - \frac{g \cdot x^2}{2 \cdot v_0^2 \cdot (\cos(\theta))^2} \quad (1) \quad \text{Ecuación de la trayectoria de un proyectil}$$
$$h = \sqrt{x^2 + y^2} \quad (2) \quad \text{Hipotenusa de un triángulo rectángulo}$$
$$x = \frac{x^\circ \cdot \pi}{180^\circ} \quad (3) \quad \text{Fórmula de conversión de grados a radianes}$$

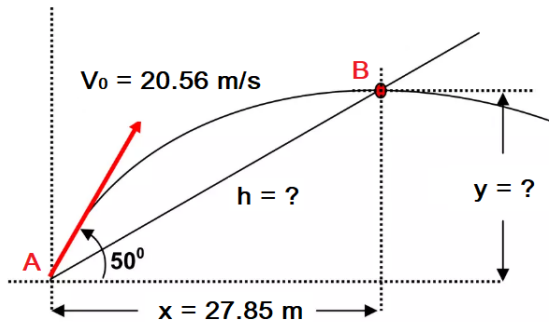


Figura 1.38.1. Tiro parabólico en un plano inclinado. (Serway, R., Jewett, J., 2005).

Por ejemplo, con los siguientes datos:

$$x = 27.85 \text{ m}; \theta = 50^\circ; V_0 = 20.56 \text{ m/s}; g = 9.80665 \text{ m/s}^2$$

Se obtienen los siguientes resultados:

$$y = 11.41 \text{ m}; h = 30.10 \text{ m}$$

Programa 1.38: Código del programa.

```
/******  
WinConsolaPrograma_1_38  
*****/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
#include <cmath>  
  
// Macros o sentencias de preprocesador.  
#define PI 3.141592  
#define g 9.80665  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // Declaración de variables.  
    float x;      // Entrada: valor de 'x' del plano inclinado.  
    float theta;  // Entrada: ángulo 'theta' de lanzamiento.  
    float Vo;     // Salida: velocidad inicial de lanzamiento.  
    float y;      // Salida: valor de 'y' del plano inclinado.  
    float h;      // Salida: valor de 'h' del plano inclinado.  
  
    // Imprimir un mensaje de información y dos saltos de línea.  
    cout << "Tiro Parabólico en un Plano Inclinado." << endl << endl;  
    // Leer el valor de 'x' del plano inclinado.  
    cout << "Ingrese el valor de 'x' del plano inclinado: ";  
    cin >> x;  
    // Leer el valor del ángulo 'theta' de lanzamiento.
```



```
cout << "Ingrese el valor del ángulo 'theta' de lanzamiento: ";
cin >> theta;
// Leer el valor del ángulo 'theta' de lanzamiento.
cout << "Ingrese el valor de la velocidad inicial de lanzamiento: ";
cin >> Vo;

// Convertir el valor del ángulo theta de grados a radianes.
theta = theta * PI / 180;
// Calcular el valor de 'y' del plano inclinado.
y = (x * tan(theta)) - (g * x * x) / (2 * Vo * Vo * cos(theta) * cos(theta));
// Calcular el valor de 'h' del plano inclinado.
h = sqrt(x * x + y * y);

// Imprimir un INTRO.
cout << endl;
// Imprimir el valor de la variable 'y' del plano inclinado.
cout << "El valor de 'y' es: " << y << endl;
// Imprimir el valor de la variable 'h' del plano inclinado.
cout << "El valor de 'h' es: " << h << " grados." << endl << endl;

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Salida 1.38: Salida del programa.

1.9. Resumen

1. El compilador de C/C++ traduce el código fuente a código objeto. El enlazador luego combina el código objeto para producir el programa ejecutable que puede correr sobre un sistema operativo.
2. Cada programa en lenguaje C/C++ tiene una función principal (main), la cual define el punto de entrada a un programa.
3. La función de salida cout permite desplegar datos desde la ventana de la consola utilizando el operador de inserción '<<'. La función de entrada cin permite la entrada de datos por consola utilizando el operador de extracción '>>'.
4. Una variable permite almacenar un elemento en la memoria de acceso aleatorio (RAM) un valor de algún tipo de dato definido en el lenguaje C/C++.
5. Los cinco operadores aritméticos binarios que maneja el lenguaje C/C++ son: suma (+), resta (-), multiplicación (*), división (/), módulo (%).
6. Los seis operadores relacionales que maneja el lenguaje C/C++ son: igual a (==), no igual a (!=), mayor que (>), menor que (<), mayor o igual que (>=), menor o igual que (<=).
7. Los tres operadores lógicos que maneja el lenguaje C/C++ son: operador lógico AND (&&), operador lógico OR (||), operador lógico de negación NOT (!).
8. La Librería de Encabezado de Funciones Matemáticas (Math Library Header), es una librería que tiene funciones para realizar operaciones matemáticas básicas, tales como: funciones trigonométricas, funciones exponenciales, funciones logarítmicas, funciones

Lenguaje C/C++. Problemas, Casos y Proyectos

para calcular la raíz cuadrada, funciones para calcular el valor absoluto de un número, etc.

9. En el lenguaje C/C++ se pueden crear programas que utilicen operadores aritméticos, relacionales y lógicos para resolver problemas de programación.

1.10. Test de Conocimientos

1. ¿Cuál es el punto de entrada de un programa en el lenguaje C/C++?
 - a. El programa comienza con la primera línea de código.
 - b. El programa comienza con la primera directiva include.
 - c. El programa comienza con función principal 'main'.
 - d. El programa comienza con el primer signo de llave '{' abierta en alguna parte del programa.
2. ¿Cuáles son los dos pasos del proceso de traducción de un programa en el lenguaje C/C++?
 - a. El enlazamiento seguido de la compilación.
 - b. La compilación seguida por el enlazamiento.
 - c. La ejecución seguida por la compilación.
 - d. El enlazamiento seguido por la ejecución.
3. El compilador traduce un archivo .CPP a un:
 - a. Archivo .OBJ
 - b. Archivo .EXE
 - c. Archivo .H
 - d. Archivo .LIB
4. ¿Cuál de las siguientes sentencias es verdadera?
 - a. Los comentarios son útiles para escribir notas y explicaciones dentro del código.
 - b. Los malos comentarios pueden ser peores que no tenerlos.
 - c. El compilador cuando compila el programa, se salta cualquier comentario y no lo toma en cuenta.
 - d. Todas las opciones anteriores.
5. Se necesita incluir la librería _____ para utilizar la función cout y la función cin y se necesita incluir la librería _____ para utilizar los tipos de datos string.
 - a. Librería stdio y conio
 - b. Librería iostream y string
 - c. Librería cstdlib y conio
 - d. Librería cstdio y cstdlib
6. El operador << se llama el operador de _____, y el operador >> se llama el operador de _____.
 - a. doble menor que, doble mayor que.
 - b. out, in.
 - c. inserción, extracción.
 - d. extracción, inserción.
7. ¿Cuál de los siguientes tipos de datos no corresponde a un tipo en C/C++?
 - a. float
 - b. integer
 - c. bool
 - d. char
8. ¿Cuál de las siguientes opciones corresponde a un nombre correcto de una variable?
 - a. 17miNombre
 - b. \$String!_GO
 - c. _valor
 - d. Int

9. ¿Cuál es el valor de la siguiente expresión?

`int x = ((3 + 2) * 5) - (12 / ((-2) * (2 + 1))) - (3 * 7) + 7;`

- a. 4
- b. 9
- c. 11
- d. 13

10. Dados los siguientes valores: A = Falso; B = Verdadero; C = Verdadero. Evalúe la siguiente expresión lógica:

`(!(6 == 10) || (2 <= 7)) && ((A || B) && C)`

- a. Verdadero
- b. Falso
- c. 6
- d. 7

11. Dados los siguientes valores: A = Verdadero; B = Falso; C = Verdadero. Evalúe la siguiente expresión lógica:

`(12 != 20) || !(A && B || C)`

- a. Falso
- b. Verdadero
- c. 12
- d. 20

12. Dado el siguiente fragmento de código:

```
int a = 7;
int b = 2;
int c;

c = a++ * b--;
a = 2 * b++;
b = 25 % a--;

cout << "a = " << a << endl;
cout << "b = " << b << endl;
cout << "c = " << c << endl;
```

- ¿Cuáles son los valores de las variables 'a', 'b' y 'c'?

- a. 2, 0, 8
- b. 4, 1, 8
- c. 1, 1, 14
- d. 4, 1, 14

13. Dado el siguiente fragmento de código:

```
int a = 7;
int b = 2;
int c;

c = (14 / a++) * (3 * b--);
c = (3 * a++) * (3 + b++);
a = (2 * b++) * (4 * a++);
b = 25 % a--;

cout << "a = " << a << endl;
cout << "b = " << b << endl;
cout << "c = " << c << endl;
```

- ¿Cuáles son los valores de las variables 'a', 'b' y 'c'?

- a. 140, 25, 96

- b. 144, 25, 98
- c. 142, 24, 98
- d. 143, 25, 96

14. Dado el siguiente fragmento de código:

```
float x = 9;
float y;
float z;

y = sqrt(x) * 3 + pow(x, 2);
z = 27 / sqrt(x) + (2 * pow(x, 3) / 3);

cout << "x = " << x << endl;
cout << "y = " << y << endl;
cout << "z = " << z << endl;
```

¿Cuáles son los valores de las variables 'a', 'b' y 'c'?

- a. 9, 96, 492
- b. 9, 90, 495
- c. 9, 96, 490
- d. 9, 98, 492

Respuestas:

1. c); 2. b); 3. a); 4. d); 5. b); 6. c). 7. b); 8. c); 9. d); 10. a); 11. b); 12. c); 13. d); 14. b).

1.11. Problemas Propuestos

1. Escribir un programa que imprima la letra A con asteriscos:

```
*****
*           *
*           *
*           *
*****
*           *
*           *
*           *
*           *
*           *
```

2. Escribir un programa que imprima la letra B con asteriscos:

```
*****
*           *
*           *
*           *
*****
*           *
*           *
*           *
*           *
*****
```

Lenguaje C/C++. Problemas, Casos y Proyectos

3. Escribir un programa que imprima la letra C con asteriscos:

```
*****
*
*
*
*
*
*
*
*
*****
```

4. Escribir un programa que imprima un rectángulo con asteriscos:

```
*****
*           *
*           *
*           *
*****
```

5. Escribir un programa que imprima un cuadrado con asteriscos:

```
****
*  *
*  *
****
```

6. Escribir un programa que imprima un triángulo con asteriscos:

```
*
**
***
****
*****
```

7. ¿Cuál es el valor de las siguientes expresiones?

- a. $16 * 17 - 4 * 8$
- b. $(25 + 3 * 7) / 5$
- c. $4 + 5 * (9 * (5 - (10 + 4) / 7))$
- d. $5 * 4 * 6 + 9 * 5 * 3 - 6$

8. Escribir las siguientes expresiones aritméticas como expresiones de computadora. Se pueden utilizar las funciones matemáticas de la librería "cmath":

- a. $e = (a + b + c) \frac{(a-b)^2}{(c-b)^2}$
- b. $z = \frac{(x+y)}{(x-y)^2}$
- c. $k = \sqrt{s \cdot (s - a) \cdot (s - b) \cdot (s - c)}$
- d. $f = \frac{9}{5} c \cdot 32$
- e. $x = \frac{c \cdot e - b \cdot f}{a \cdot e - b \cdot d}$
- f. $F = \frac{G \cdot m_1 \cdot m_2}{d^2}$
- g. $h = \sqrt{a^2 + b^2}$
- h. $e = m \cdot c^2$

9. Escribir un programa que permita representar la expresión algebraica: $(A \&\& B) \parallel (! A \&\& ! B)$ y probar una de las salidas de la puerta lógica XNOR, para lo cual se debe leer por teclado el valor de verdad de A y de B, utilizando las funciones 'cout' y 'cin'. Esta puerta lógica se conoce también como el bicondicional $(A \leftrightarrow B)$, cuya tabla de verdad es:

A	B	$A \leftrightarrow B$
V	V	V
V	F	F
F	V	F
F	F	V

10. Escribir un programa que permita representar la expresión algebraica: $!(A \parallel B)$ y probar una de las salidas de la puerta lógica NOR, para lo cual se debe leer por teclado el valor de verdad de A y de B, utilizando las funciones 'cout' y 'cin'. Esta puerta lógica se conoce también como la disyunción opuesta $(A \downarrow B)$, cuya tabla de verdad es:

A	B	$A \downarrow B$
V	V	F
V	F	F
F	V	F
F	F	V

11. Escribir un programa que permita representar la expresión algebraica: $!(A \&\& B)$ y probar una de las salidas de la puerta lógica NAND, para lo cual se debe leer por teclado el valor de verdad de A y de B, utilizando las funciones 'cout' y 'cin'. Esta puerta lógica se conoce también como la conjunción opuesta $(A \uparrow B)$, cuya tabla de verdad es:

A	B	$A \uparrow B$
V	V	F
V	F	V
F	V	V
F	F	V

12. Escribir un programa que permita representar la expresión algebraica: $!A \parallel B$ y probar una de las salidas de esta puerta lógica, para lo cual se debe leer por teclado el valor de verdad de A y de B, utilizando las funciones 'cout' y 'cin'. Esta puerta lógica se conoce como la implicación lógica $(A \rightarrow B)$, cuya tabla de verdad es:

A	B	$A \rightarrow B$
V	V	V
V	F	F
F	V	V
F	F	V

13. Escribir un programa que permita representar la expresión algebraica: $A \&\& !B$ y probar una de las salidas de esta puerta lógica, para lo cual se debe leer por teclado el valor de verdad de A y de B, utilizando las funciones 'cout' y 'cin'. Esta puerta lógica se conoce como la adjunción lógica $(A \nrightarrow B)$, cuya tabla de verdad es:

A	B	$A \nrightarrow B$
V	V	F
V	F	V
F	V	F
F	F	F

14. Escribir un programa que permita representar la expresión algebraica: $A \parallel !B$ y probar una de las salidas de esta puerta lógica, para lo cual se debe leer por teclado el valor de

Lenguaje C/C++. Problemas, Casos y Proyectos

verdad de A y de B, utilizando las funciones 'cout' y 'cin'. Esta puerta lógica se conoce como la implicación opuesta ($A \leftarrow B$), cuya tabla de verdad es:

A	B	$A \leftarrow B$
V	V	V
V	F	V
F	V	F
F	F	V

15. Escribir un programa que permita representar la expresión algebraica: $!A \&\& B$ y probar una de las salidas de esta puerta lógica, para lo cual se debe leer por teclado el valor de verdad de A y de B, utilizando las funciones 'cout' y 'cin'. Esta puerta lógica se conoce como la adjunción opuesta ($A \leftrightarrow B$), cuya tabla de verdad es:

A	B	$A \leftrightarrow B$
V	V	F
V	F	F
F	V	V
F	F	F

16. Escribir un programa que solicite al usuario que ingrese los valores de los dos catetos 'a' y 'b' de un triángulo rectángulo (trabajar con valores flotantes), para calcular el valor de la hipotenusa, el perímetro y el área de esta figura, utilizando las siguientes fórmulas matemáticas:

$$\begin{aligned} h &= \sqrt{a^2 + b^2} & (1) & \text{Hipotenusa del triángulo} \\ p &= a + b + h & (2) & \text{Perímetro del triángulo} \\ \text{área} &= \frac{a \cdot b}{2} & (3) & \text{Área del triángulo} \end{aligned}$$

17. Escribir un programa que solicite al usuario que ingrese el precio de un artículo y se calcule el precio total incluido el impuesto del 12%. (Trabajar con valores flotantes).

Ejemplo:

Ingrese el precio de un artículo: 60

El precio total incluido el impuesto del 12% es: 67.2

18. Escribir un programa que solicite al usuario que ingrese el largo y el ancho un cuarto en metros y despliegue el área del piso del cuarto y el costo de alfombrar el mismo, considerando que el costo del metro cuadrado es de \$15 USD. (Trabajar con valores flotantes)

Ejemplo:

Ingrese el ancho del cuarto: 5

Ingrese el largo del cuarto: 4

El área del piso es: 20

El costo de la alfombra es: 300

19. Escribir un programa que lea la temperatura en grados Celsius y la escriba en grados Fahrenheit (trabajar con valores flotantes), utilizando la siguiente fórmula matemática:

$$f = \frac{9}{5}c + 32 \quad (1) \quad \text{Fórmula de conversión de grados Celsius a Fahrenheit}$$

20. Escribir un programa que permita convertir un punto **P** dado en coordenadas cartesianas a coordenadas polares, donde este punto **P** siempre va a estar en el tercer cuadrante del plano cartesiano. Así, por ejemplo, el punto $P(x, y)$, se convierte en: (r, θ) , utilizando las siguientes fórmulas matemáticas (ver Figura 1.11.1). Además, se debe tomar en cuenta que el ángulo theta (θ) obtenido hay que convertirlo de radianes a grados, de

Lenguaje C/C++. Problemas, Casos y Proyectos

acuerdo a las funciones trigonométricas de la librería <cmath>. (Problema adaptado de Serway y Jewett, 2005; pp. 54; Ejemplo 3.1).

$$r = \sqrt{x^2 + y^2} \quad (1)$$

$$\theta = \arctan\left(\frac{y}{x}\right) + \pi \quad (2)$$

$$x = \frac{x^\circ \cdot 180}{\pi} \quad (3)$$

Fórmula para obtener el valor del radio de un punto.

Fórmula para obtener el ángulo de inclinación del radio.

Fórmula para convertir de radianes a grados.

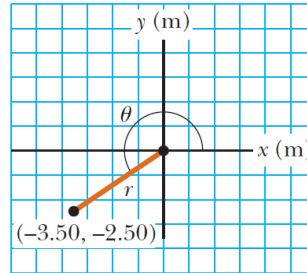


Figura 1.11.1. Convertir coordenadas rectangulares a polares. (Serway y Jewett, 2005; pp. 54; Ejemplo 3.1).

Por ejemplo, con los siguientes datos:

$$P(x, y) = (-3.50, -2.50)m$$

Se obtienen los siguientes resultados:

$$(r, \theta) = (4.30, 35.54)$$

21. Escribir un programa para encontrar la magnitud y dirección del desplazamiento resultante de un automóvil ($\vec{R} = \vec{A} + \vec{B}$), asumiendo que este vehículo viaja primero al norte una distancia **A** en Km y luego una distancia **B** en Km en una dirección α , al noroeste, como se muestra en la Figura 1.11.2, utilizando las siguientes fórmulas matemáticas. Además, se debe tomar en cuenta que el ángulo alpha (α) se lee en grados y luego se convierte a radianes, el ángulo theta (θ) se calcula en grados y luego se convierte a radianes y finalmente el ángulo beta (β) obtenido hay que convertirlo de radianes a grados, de acuerdo a las funciones trigonométricas de la librería <cmath>. (Problema adaptado de Serway y Jewett, 2005; pp. 58; Ejemplo 3.2).

$$\theta = 180^\circ - \alpha \quad (1)$$

$$R = \sqrt{A^2 + B^2 - 2 \cdot A \cdot B \cdot \cos(\theta)} \quad (2)$$

$$\beta = \arcsen\left(\frac{B \cdot \sin(\theta)}{R}\right) \quad (3)$$

$$x = \frac{x^\circ \cdot \pi}{180^\circ}$$

$$x = \frac{x[\text{rad}] \cdot 180}{\pi}$$

Fórmula para obtener el valor del ángulo theta (θ).

Fórmula para obtener el valor del vector resultante.

Fórmula para obtener el valor del ángulo betha (β).

Fórmula para convertir de grados a radianes.

Fórmula para convertir de radianes a grados.

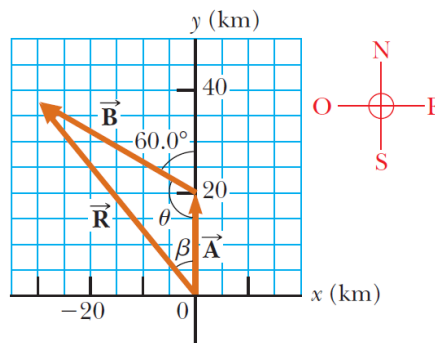


Figura 1.11.2. Método gráfico para encontrar el vector de desplazamiento resultante $\vec{R} = \vec{A} + \vec{B}$. (Serway y Jewett, 2005; pp. 58; Ejemplo 3.2)

Lenguaje C/C++. Problemas, Casos y Proyectos

Por ejemplo, con los siguientes datos:
 $A = 20 \text{ Km}; B = 35 \text{ Km}; \alpha = 60^\circ$

Se obtienen los siguientes resultados:
 $\theta = 120^\circ; R = 48.22 \text{ Km}; \beta = 38.95^\circ$

22. Escribir un programa para automatizar el cálculo de la aceleración radial, de la aceleración total y el cálculo del ángulo Phi (ϕ), formado por las aceleraciones tangencial y total, considerando el siguiente problema donde un automóvil muestra una aceleración constante de 0.40 m/s^2 paralela a la autopista. El automóvil pasa sobre una elevación en el camino tal que lo alto de la elevación tiene forma de círculo con 450 m de radio. En el momento en que el automóvil está en lo alto de la elevación, su vector velocidad es horizontal y tiene una magnitud de 7.00 m/s (ver la Figura 1.11.3). ¿Cuáles son la magnitud y dirección del vector aceleración total para el automóvil en ese instante? Utilizar las siguientes fórmulas matemáticas. (Problema adaptado de Serway y Jewett, 2005; pp. 87; Ejemplo 4.7).

$$a_r = -\frac{v^2}{r}$$

$$a = \sqrt{a_r^2 + a_t^2}$$

$$\phi = \arctan\left(\frac{a_r}{a_t}\right)$$

$$x = \frac{x[\text{rad}] \cdot 180}{\pi}$$

- (1) Fórmula de la aceleración radial.
- (2) Fórmula de la aceleración total.
- (3) Fórmula del ángulo Phi formado por la aceleración radial y la tangencial.
- (4) Fórmula para convertir de radianes a grados.

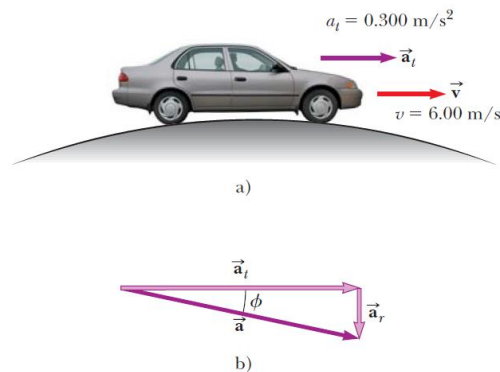


Figura 1.11.3. a) Un automóvil pasa sobre una elevación que tiene forma de círculo. b) El vector aceleración $\vec{a}$ total que es la suma de los vectores aceleración tangencial $\vec{a}_t$ y radial $\vec{a}_r$. (Serway y Jewett, 2005; pp. 87; Ejemplo 4.7).

Por ejemplo, con los siguientes datos:

$$a_t = 0.30 \text{ m/s}^2; v = 6.00 \text{ m/s}; r = 500 \text{ m}$$

Se obtienen los siguientes resultados:

$$a_r = -0.072 \text{ m/s}^2; a = 0.309 \text{ m/s}^2; \phi = -13.5^\circ$$

23. Escribir un programa para automatizar la ley de Gravitación Universal, considerando dos masas m_1 y m_2 en Kg, que se encuentran separadas una distancia d en metros. Calcular la fuerza de atracción que experimenta las masas, utilizando la siguiente fórmula matemática. (Problema adaptado de Fisimat, s.f.; Problema 1).

$$F_g = G \frac{m_1 \cdot m_2}{r^2} \quad (1) \text{ Fórmula para obtener la fuerza de atracción entre dos masas.}$$

$$G = 6.673 \times 10^{-11} \text{ N} \cdot \text{m}^2/\text{Kg}^2 \quad (2) \text{ Constante Gravitacional Universal}$$

Por ejemplo, con los siguientes datos:

$$m_1 = 800 \text{ Kg}; m_2 = 500 \text{ Kg}; d = 3 \text{ m}$$

Se obtienen los siguientes resultados:

$$F = 2.966 \times 10^{-6} \text{ N}$$

24. Escribir un programa para automatizar la fórmula de Einstein, considerando que un Kilogramo de Uranio reacciona en una bomba atómica transformando 0.915 g de su masa en energía. Como la masa viene dada en gramos se va a utilizar los ergios como unidad de medida de la energía, donde 1 ergio es igual a $1\text{ g} \cdot \text{cm}^2/\text{s}^2$. Además, se va a pasar la velocidad de la luz a cm/s , por lo que queda $3 \times 10^{10}\text{ cm/s}$. Calcular el valor de la energía producida por la transformación de una masa dada en gramos a energía dada en ergios, utilizando la siguiente fórmula matemática. (Problema adaptado de Docsity, s.f.; Problema 1).

$$e = mc^2 \quad (1) \quad \text{Fórmula de Einstein.}$$

Por ejemplo, con los siguientes datos:

$$m = 0.915\text{ g}; c = 3 \times 10^{10}\text{ cm/s}$$

Se obtienen los siguientes resultados:

$$e = 8.235 \times 10^{20}\text{ ergios}$$

Referencias Bibliográficas

- Ceballos, F.J., 2007. C/C++: Curso de Programación. Tercera Edición. Editorial RA-MA, España. ISBN: 978-84-9964-322-9.
- Davies, P., 1995. The Indispensable Guide to C with Engineering Applications. First Edition. Addison-Wesley Publishing Company, USA. ISBN: 0-201624389.
- Deitel, H., Deitel, P., 2003. Cómo Programar en C++. Cuarta Edición. Pearson Educación, México. ISBN: 970-26-0254-8.
- Docsity (s.f.). Teoría de la relatividad. Sitio Web Educativo. Disponible en: <https://www.docsity.com/es/teoria-de-la-relatividad-fisica-ejercicios/7482343/>.
- Fisimat, (s.f.). Ley de gravitación universal – Ejercicios resueltos. Sitio Web Educativo: Física y Matemáticas. Disponible en: <https://www.fisimat.com.mx/ley-de-la-gravitacion-universal/>.
- Granizo, E., 2016. Lenguaje C. Teoría y Ejercicios. Tercera Edición. Edicumbre Editorial Corporativa, Ecuador. ISBN: En Trámite.
- Guillen, M., 2000. Cinco ecuaciones que cambiaron el mundo: El poder y la belleza de las matemáticas. Editorial Debate S.A., Madrid, España. ISBN:9788483062289.
- Jones, B., Aitken, P., 2003. C in 21 Days. Sixth Edition. SAMS Teach Yourself, USA. ISBN: 0-672-32448-2.
- Joyanes Aguilar, L., Zahonero Martínez, I., 2005. Programación en C. Metodología, algoritmos y estructuras de datos. Segunda Edición. McGraw-Hill, España. ISBN: 978-8448198442.
- Luna, F., 2004. C++ Programming for Game Developers. Module I. Game Institute. E-Institute, Inc., USA.
- Ritchie, D., Kernighan, B., 1988. C Programming Language. Second Edition. Prentice Hall, USA. ISBN: 0-13-110362-8.
- Serway, R., Jewett, J., 2005. Física para Ciencias e Ingeniería. Volumen 1. Séptima Edición. ISBN: 978-607-481-357-9. Cengage Learning Editores, México.
- Serway, R., Jewett, J., 2009. Física para Ciencias e Ingeniería. Volumen 2. Séptima Edición. ISBN: 978-607-481-358-6. Cengage Learning Editores, México.
- Stroustrup, B., 1997. The C++ Programming Language. Special Edition. Addison-Wesley, USA. ISBN: 0-201-88954-4.
- Zambrano, M., Villacís, C., Alvarado, D., Pérez, D., Carvajal, V., Guijarro, J., Prajapati, Neerav., Sunday-Oyelere, S., 2021. Active learning of programming as a complex technology applying problem solving, programming case study and OnlineGDB Compiler. 10th International Conference on Educational and Information Technology (ICEIT). DOI: 10.1109/ICEIT51700.2021.9375611. ISBN:978-1-6654-2295-6.